IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 23 May 2023, accepted 28 May 2023, date of publication 1 June 2023, date of current version 7 June 2023.

Digital Object Identifier 10.1109/ACCESS.2023.3281910

RESEARCH ARTICLE

Impact of Deep Learning Optimizers and Hyperparameter Tuning on the Performance of Bearing Fault Diagnosis

SEONGJAE LEE AND TAEHYOUN KIM, (Member, IEEE)
Department of Mechanical and Information Engineering/Smart Cities, University of Seoul, Seoul 02504, South Korea

Corresponding author: Taehyoun Kim (thkim@uos.ac.kr)

This work was supported by the National Research Foundation of Korea (NRF) Grant funded by the Korea Government [Ministry of Science and ICT (MSIT)] under Grant 2022R1F1A1060231.

ABSTRACT Deep learning has recently resulted in remarkable performance improvements in machine fault diagnosis using only raw input vibration signals without signal preprocessing. However, research on machine fault diagnosis using deep learning has primarily focused on model architectures, even though optimizers and their hyperparameters used for training can have a significant impact on model performance. This paper presents extensive benchmarking results on the tuning of optimizer hyperparameters using various combinations of datasets, convolutional neural network (CNN) models, and optimizers with varying batch sizes. First, we set the hyperparameter search space and then trained the models using hyperparameters sampled from a quasi-random distribution. Subsequently, we refined the search space based on the results of the first step and finally evaluated model performances using noise-free and noisy data. The results showed that the learning rate and momentum factor, which determine training speed, substantially affected the model's accuracy. We also discovered that the impacts of batch size and model training speed on model performance were highly correlated; large batch sizes led to higher performances at higher learning rates or momentum factors. Conversely, model performances tended to be high for small batch sizes at lower learning rates or momentum factors. In addition, regarding the growing attention to on-device artificial intelligence (AI) solutions, we assessed the accuracy and computational efficiency of candidate models. A CNN with training interference (TICNN) was the most efficient model in terms of computational efficiency and robustness against noise among the benchmarked candidate models.

INDEX TERMS Bearing fault diagnosis, convolutional neural network, deep learning, hyperparameter tuning, noise-robustness, optimization.

I. INTRODUCTION

In modern industries, rolling element bearings (REBs) are an essential part of rotating machinery operations, and 40–50% of equipment failures are caused by damaged REBs [1]. Because such device defects can lead to economic loss or loss of life, research on condition monitoring of REBs has continuously gained attention. Many machine condition monitoring studies have performed fault detection using data-driven vibration analysis.

The associate editor coordinating the review of this manuscript and approving it for publication was Mehrdad Saif.

The traditional data-based fault detection approach comprises four stages [2]: data acquisition, feature extraction, feature selection, and classification. The vibration signal of a rotating machine can be acquired using an accelerometer, and the sampling rate and resolution can vary depending on the performance of the accelerometer. Data characteristics are extracted in the feature extraction step by converting the collected time domain data into frequency or time-frequency domain data. Fast Fourier transform (FFT) is a representative method for converting time domain data into frequency domain data, and short-time Fourier transform (STFT) and wavelet transform (WT) are also commonly used for

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

generating time-frequency domain data from time domain data. In the feature selection step, techniques, such as principal component analysis [3], [4], [5], extract latent features from high-dimensional data. Finally, in the classification stage, machine learning (ML) methods, such as support vector machine (SVM) [4], [6], random forest (RF) [7], [8], and k-nearest neighbor (kNN) [9], [10], are used to classify machine faults. However, the disadvantage of the traditional approach is that the feature extraction and selection steps, depending on the domain expertise, may affect the fault detection performance.

Recently, deep learning has achieved remarkable performance improvements in various fields, such as image processing, natural language processing, and time series forecasting [11]. In particular, deep learning models achieve higher performance than conventional data-based approaches even when raw inputs are used without feature extraction and selection [12]. Furthermore, in bearing fault diagnosis, many researchers have obtained approximately 100% accuracy without preprocessing using convolutional neural networks (CNN) [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], recurrent neural networks (RNN) [28], [29], [30], and Transformers [31], [32]. Studies have also been conducted to diagnose the failures of multi-domain data measured under different working loads or to develop a model that can classify bearing faults even in a noisy environment. Deep learning-based approaches have also demonstrated remarkable achievements in analyzing non-stationary signals with noise in other areas, such as speech processing [33] and gravitational wave analysis [34].

Deep learning models are generally trained using a stochastic optimization method [35], and the training speed and final model performance vary significantly depending on the optimizer type and hyperparameter tuning. However, there is no theoretical basis for determining the best optimizer and hyperparameter tuning method, and empirical studies have been conducted only in representative fields of deep learning [36]. In particular, although research on model architectures has been intensively conducted in bearing fault diagnosis, there are fewer studies on the theoretical or empirical verification of the effects of optimizers and hyperparameter tuning on the model performance. Furthermore, in model development studies, the description of the optimization methods and training environments used in them is often unclear.

Regarding these problems, this paper presents a bearing fault diagnosis benchmarking study of robustness against noisy data using two open-access datasets, seven CNN-based models, and four deep learning optimizers with varying batch sizes. The benchmarking and performance verification processes were as follows. First, we set the hyperparameter search space for each optimizer. We used stochastic gradient descent (SGD), Momentum, RMSProp, and Adam as the optimizers. The hyperparameter set for each optimizer was sampled from a quasi-random log-uniform distribution. Further, we trained a model on each hyperparameter set and refined the hyperparameter search space for each model and optimizer. Finally, after training the models in the refined hyperparameter search space, we evaluated the results using noise-free and noisy data.

This paper demonstrates that the hyperparameter of the optimizer, which has been underestimated in deep learning-based bearing fault diagnosis, significantly affects the robustness of the models against noise using a comprehensive benchmark with various models and datasets. We also conducted an extensive performance comparison of various CNN-based models regarding noise-robustness, model parameters, and computational efficiency in a refined search space where each model and dataset achieved high accuracy, considering on-device artificial intelligence (AI). These findings can help researchers efficiently design deep learning-based fault diagnosis models and choose hyperparameters for model training. Furthermore, to promote further exploration, we open our source code, which can be used to benchmark various user-defined models, datasets, and hyperparameters.

The remainder of this paper is organized as follows. Section II introduces related studies on bearing fault diagnosis, deep learning optimizer evaluation, and benchmarking studies. In Section III, we describe our benchmarking approach and the datasets, models, optimizers, and hyperparameters used in the experiments. Section IV presents the benchmarking results and a review of the results. Finally, we summarize and conclude the paper in Section V.

II. RELATED WORK

A. DEEP LEARNING-BASED BEARING FAULT DIAGNOSIS

Deep learning models have attracted considerable interest in fault diagnosis because they can achieve high performance without prior feature extraction. Among them, the CNN is the most commonly used structure, and 1D and 2D CNNs have been widely studied. The 1D CNN receives raw vibration signal inputs to diagnose device failures. Zhang et al. [13], [14] proposed a model that maximizes the robustness against noise by widening the kernel size of the first convolutional layer. Recently, there have also been active studies on fault diagnosis based on 1D CNNs that employ structures such as residual connections and dilated convolutions, which have been proven to perform well in other application areas, including image processing and audio processing [15], [16], [17], [18], [19], [20]. On the other hand, 2D CNN-based algorithms incorporate various feature extraction techniques, such as signal-to-image mapping [21], [22], [23], STFT [24], [25], cyclic spectral coherence [26], and nonlinear mode decomposition [27].

RNNs, in conjunction with CNNs, are also widely used in fault diagnosis. For example, input signals are first passed through a CNN, and then a RNN diagnoses failures through the signals. Shenfield and Howarth [28] proposed a hybrid model of long short-term memory (LSTM) and a CNN-based model described in [13]. In addition, Jin et al. [29] proposed an adaptive anti-noise neural network framework using a

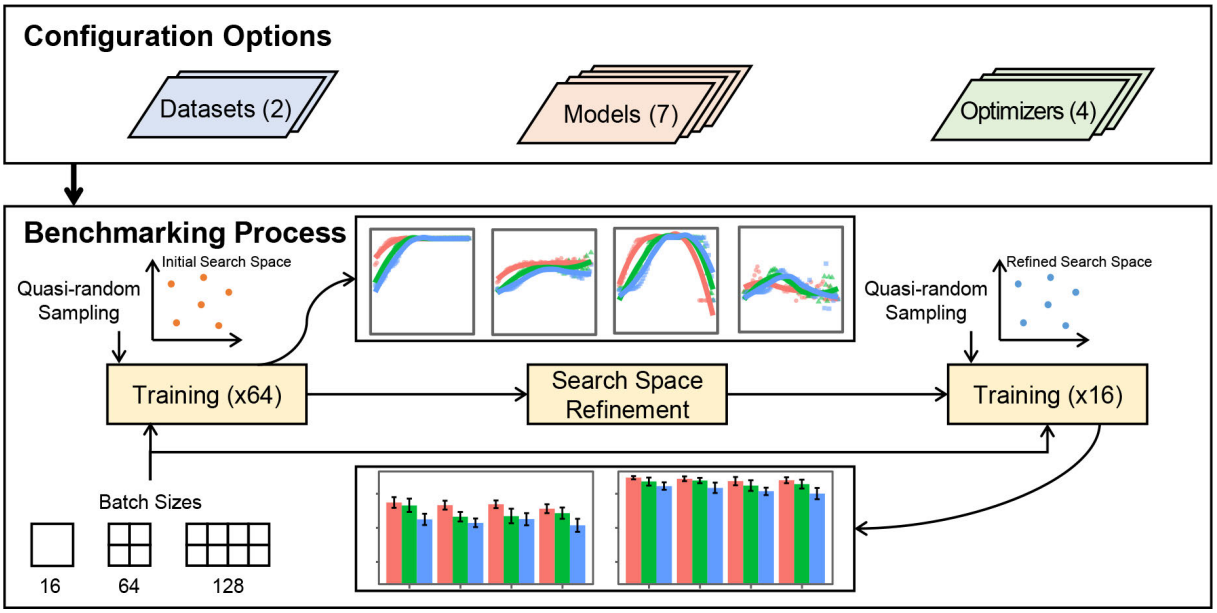


FIGURE 1. Schematic workflow of the benchmarking process.

1D CNN, a gate recurrent unit, and an attention module. Currently, Transformer-based architectures have also been applied to fault diagnosis. For example, Feng et al. [31] presented a lightweight combination model of a 1D CNN and a Transformer structure, and Ding et al. [32] proposed a Transformer structure that processes time-frequency domain data.

B. DEEP LEARNING OPTIMIZERS AND HYPERPARAMETER TUNING

Most deep learning models are trained by stochastic optimization, which minimizes the loss from data in mini-batch units of a specific size. The most basic and widely used optimization algorithm for model training is SGD. The Momentum algorithm [37] was devised to improve the slow learning speed of SGD. Currently, the RMSProp [38] and Adam [39] algorithms are widely used because they adaptively change the learning rate during the training process.

The importance of properly tuning the optimizers and their hyperparameters in deep learning model training has also been emphasized. Grid search is the simplest and most basic hyperparameter tuning method. It is an exhaustive method for selecting the best hyperparameter by evaluating the performance of all the combinations. Although this method is simple and easy to implement and parallelize, its search time increases exponentially even if the hyperparameter dimension increases only slightly. Random search is a widely used hyperparameter tuning method owing to its high efficiency [36], [40], [41], [42]. It randomly samples hyperparameter combinations within a predetermined range.

Research on the hyperparameter tuning of deep learning optimizers has been conducted in various fields. However, there is no theoretically verified best optimizer or hyperparameter tuning method [42], and benchmarking and empirical studies on specific datasets and models are predominant. Choi et al. [36] presented large-scale optimization hyperparameter tuning results for image classification and language modeling tasks. Schmidt et al. [42] also presented the hyperparameter tuning results for each optimizer in image classification, image generation, and character prediction tasks.

In addition to large-scale benchmarking studies, studies have been conducted to verify the effectiveness of deep learning optimizers and hyperparameter tuning for a specific target application. For example, Verma et al. [43] verified the performance of Adam and RMSProp optimizers based on a learning rate change in a COVID-19 classification problem using computed tomography images. Saleem et al. [44] compared the performances of six optimizers, including SGD and Adam, for CNN models applied to plant disease classification. Şen et al. [45] presented the results of a grid search for the hyperparameters of the Adam optimizer in electrocardiogram classification. With regard to fault diagnosis, Rezaeianjouybari and Shang [46] verified the effects of SGD and Momentum's cyclic learning rate and cyclic momentum on bearing fault detection.

III. METHODOLOGY

This paper presents the benchmarking results of the impact of hyperparameter tuning on deep learning optimizers with noisy vibration data. Fig. 1 depicts the entire process of hyperparameter tuning and benchmarking performed in this study. We benchmarked 56 configuration options from combinations of two open datasets, seven CNN models, and four optimizers. The deep-learning model in each configuration was trained for batch sizes of 16, 64, and 128.

As the first step of our benchmarking study, we specified the search space of the optimizer for each hyperparameter and

performed model training 64 times. In this step, hyperparameters were sampled from a quasi-random uniform distribution on a log scale. Second, we refined the search space for each model and optimizer based on the experimental results acquired from the first step and then repeated the model training 16 times. Based on the experimental results from the second step, we evaluated the test accuracies of the noise-free and noisy data for different batch sizes and optimizers.

A. DATASETS

This benchmarking study used the Case Western Reserve University (CWRU) and Society for Machinery Failure Prevention Technology (MFPT) datasets. Both datasets are widely used in bearing fault diagnosis and provide vibration signals for various bearing faults and motor load environments.

The CWRU dataset [47] is the most widely used public dataset for bearing fault diagnosis. This dataset contains vibration data measured using an accelerometer with 12 and 48 kHz sampling rates from a 2 HP reliance electric motor. Fig. 2 shows the test rig in which the dataset was collected. Two types of bearings were installed at the test rig: a 6205-2RS JEM SKF bearing at the drive end and a 6203-2RS JEM SKF bearing at the fan end. Data were collected for each bearing's inner raceway, outer raceway, and ball fault situations. In addition, bearing faults were artificially synthesized using the electro-discharge machining method to include data with various crack sizes and motor loads. Our study used the drive end bearing data collected under 0.7457, 1.4914, and 2.2371 kW motor loads at 12 kHz.

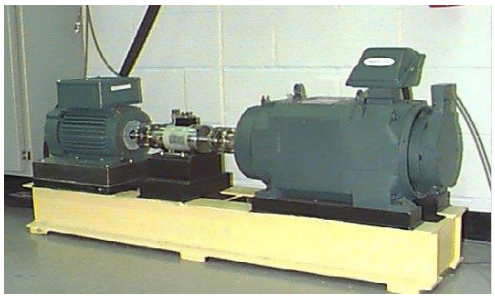


FIGURE 2. Test rig of the CWRU dataset [47].

The MFPT dataset [48] provides vibration data measured for normal bearings and two types of defective bearings: inner and outer raceway faults. The vibration data were collected under nine load conditions at a sampling rate of 97,656 Hz.

Both bearing datasets consist of a small number of long-duration raw vibration signals measured over a predefined time interval under specific bearing conditions. For example, the inner raceway fault data of the CWRU dataset are composed of nine raw signals, and the length of each signal is approximately 120,000. If these raw signals are used for training directly, the size and computational burden of a model increase, whereas the number of the training data samples is insufficient. Therefore, we regenerated the data using the following three steps: data labeling, splitting, and augmentation.

First, we labeled the data according to the bearing fault type. Because the CWRU dataset provided fault data for three different crack sizes and three fault types, the data were labeled with nine fault classes and a normal class. Table 1 presents the class classification method for the CWRU dataset. However, the MFPT dataset provided only two types of faults, inner and outer raceways, and did not separate the detailed characteristics of each fault. Therefore, the data were classified into three classes: normal, inner raceway fault, and outer raceway fault. Table 2 shows the class classification method for the MFPT dataset. Subsequently, we split each vibration signal into training, validation, and testing sets with a ratio of 60%:20%:20%, respectively.

Finally, we generated the data by overlapping and slicing the raw signals, which have often been used to secure sufficient training data, as shown in Fig. 3. We sliced the data with the largest input size of the candidate models, 4,096, and a shift size of 2,048. However, the models evaluated in this study have different input sizes, and adjusting the input sizes of each model requires a modification of the model architecture. Because it may change the performance of the models, we did not modify the original input size of the models. When the input size of the model was smaller than 4,096, only the front part of the data was used to train each model using the same data. For example, a model with an input size of 2,048 was trained using only the first 2,048 elements of data. Tables 1 and 2 also list the number of samples for each dataset used in the benchmarking study.

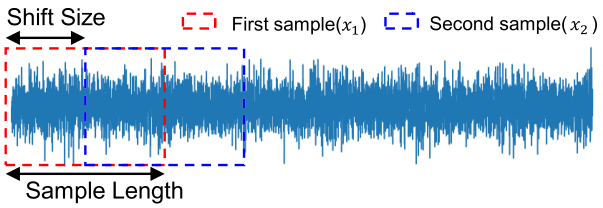


FIGURE 3. Data augmentation using overlapping.

TABLE 1. Data labels and the number of samples for the CWRU dataset.

Fault Type	Fault Diameter (mm)	Class Label	Number of Samples		
			Train	Validation	Test
Normal	-	0	422	138	138
Ball	0.1778	1	102	30	30
	0.3556	2	102	30	30
	0.5334	3	102	30	30
Inner Raceway	0.1778	4	103	31	31
	0.3556	5	102	30	30
	0.5334	6	102	30	30
Outer Raceway	0.1778	7	102	30	30
	0.3556	8	102	30	30
	0.5334	9	102	30	30

B. CANDIDATE MODELS

Seven candidate models were employed in our benchmarking study: four 1D CNNs, two 2D CNNs, and one CNN + RNN model. Table 3 summarizes the candidate models and their concepts.

TABLE 2. Data labels and the number of samples for the MFPT dataset.

Fault Type	Class Label	Number of Samples		
		Train	Validation	Test
Normal	0	510	168	168
Inner Raceway	1	287	91	91
Outer Raceway	2	797	259	259

TABLE 3. Summary of candidate models.

Category	Model	Concept
1D CNN	WDCNN [13]	• Wide First Convolutional Layer
	TICNN [14]	• Wide First Convolutional Layer • Convolutional Kernel Dropout
	DCN [15]	• Wide First Convolutional Layer • Dilated Convolution [49] • Residual Connection [50] • Squeeze-and-Excitation Block [51]
	SRDCNN [16]	• Wide First Convolutional Layer • Dilated Convolution [49] • Residual Connection [50] • WaveNet-like Architecture [52]
2D CNN	STIM-CNN [21]	• Signal-to-Image Mapping
	STFT-CNN [24]	• STFT
1D CNN + RNN	RNN-WDCNN [28]	• Integration of WDCNN and LSTM

Because vibration signals are one-dimensional, 1D CNNs are the most popular models for bearing fault diagnosis. The main idea of early 1D CNN-based architectures was to widen the first kernel size of the convolutional layer. This technique has been broadly applied in many studies of 1D CNN-based fault detection. We used deep convolutional neural networks with wide first-layer kernels (WDCNN) [13] and TICNN [14] as representatives of the 1D CNN model.

WDCNN [13] is an early 1D CNN-based model that enhances domain adaptation and anti-noise ability for bearing fault diagnosis. This model showed that widening the kernel size of the first convolutional layer could suppress the high-frequency noise of vibration signals.

TICNN [14] is the successor of WDCNN. This model has a similar architecture to the WDCNN but uses kernel dropout for the first convolutional layer. Kernel dropout randomly drops the value of the convolutional filters. The study mentioned that the kernel dropout of the first layer could add noise to the input data. Fig. 4 shows the kernel dropout for a 1D convolutional filter of size three.

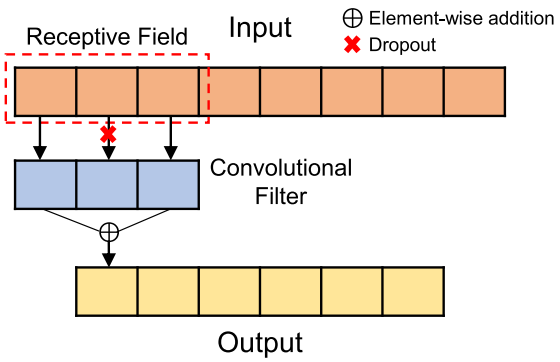


FIGURE 4. Kernel dropout of TICNN.

Dilated convolution and residual connection are also frequently used in fault detection CNN. Dilated convolution was originally proposed for image segmentation [49]. It can increase the receptive field of a convolutional kernel while keeping the parameter constant by creating holes in the convolutional kernel. Fig. 5(a) illustrates a 1D dilated convolution of a 3×1 filter. The receptive field size of this filter was five while only using the parameters of the 3×1 filter.

Residual connection adds the input layer values to the output layer values of a neural network by element. This technique can address the accuracy degradation with respect to the depth of the neural network. Fig. 5(b) shows a residual connection applied to a convolutional layer. Beginning with ResNet [50] for image classification, numerous modern CNN architectures have used the residual connection. In this study, we used a one-dimensional dilated convolution network with residual connection (DCN) and a stacked residual dilated convolutional neural network (SRDCNN) as representatives using dilated convolution and residual connection.

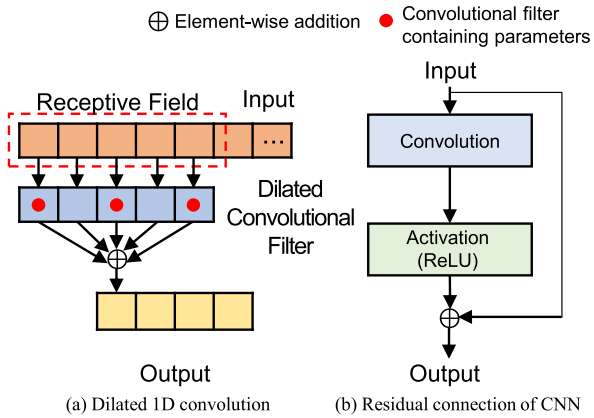


FIGURE 5. Dilated convolution and residual connection.

DCN [15] used dilated convolution and residual connection to enhance the feature learning ability. The essential blocks of DCN are the residual and dilated residual connection blocks. The residual connection block is a combination of a normal convolutional layer and residual connection, and the dilated residual connection block is composed of dilated convolutional layers with a residual connection. In addition, the squeeze-and-excitation block, proposed by SeNet [51], is used for the DCN architecture. The DCN effectively diagnosed the faults of two bearing datasets in multi-domain and noisy environments.

SRDCNN also used both residual connection and dilated convolution. The building block of the SRDCNN is similar to that of the WaveNet [52] architecture, which is based on the input gate layer of the LSTM and dilated convolution. Fig. 6 shows the building block of SRDCNN.

2D CNN architectures have also been intensively studied. Unlike 1D CNNs, 2D CNNs must convert inputs to 2D data because raw vibration signals are one-dimensional time-series data.

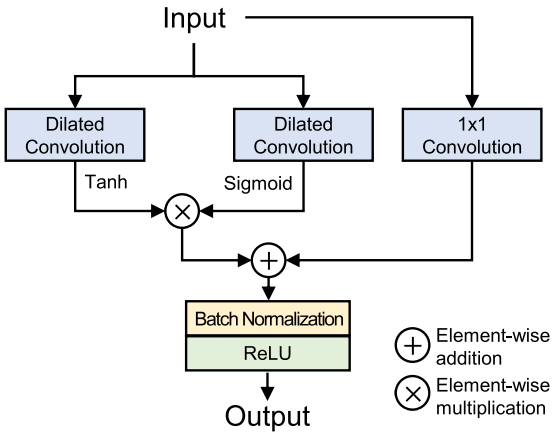
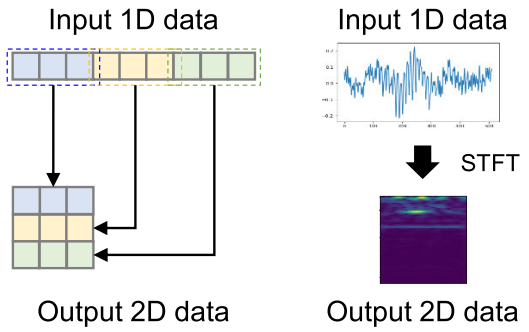


FIGURE 6. Building block of SRDCNN.

Signal-to-image mapping (STIM) is the simplest data conversion algorithm that reshapes a 1D signal to a 2D vibration image. Fig. 7(a) illustrates the STIM algorithm, which converts a 9×1 1D signal to a 3×3 2D signal. Among the previous studies on 2D fault diagnosis CNNs using STIM [21], [23], we selected the simple 2D CNN by Zhao et al. [21] (STIM-CNN) for the candidate model. The STIM-CNN had two convolutional layers and a fully connected layer that achieved higher accuracy with fewer parameters than LeNet-5.

Signal processing algorithms, such as STFT and WT, can also be used to generate 2D inputs [24], [25], [27]. These algorithms require additional computational resources to derive the time-frequency domain information. Fig. 7(b) shows a 2D spectrogram generated by the STFT. This study used the model proposed by Zhang et al. [24] (STFT-CNN) as another 2D CNN candidate. The STFT-CNN used a 2D spectrogram from STFT as inputs, and the scaled exponential linear unit was used to avoid dead nodes.



(a) STIM method (b) STFT

FIGURE 7. Data generation for 2D CNN.

Because vibration signals are a time series, RNNs have been actively applied to diagnose bearing faults [28], [29]. As the candidate model for the RNN-based architecture, we selected the RNN-WDCNN [28] as shown in Fig. 8. The model consists of two paths. One path passes the input to the WDCNN [13] directly. In another path, the input passes the convolutional layer and the LSTM layer sequentially. Finally, the two outputs from the paths are concatenated.

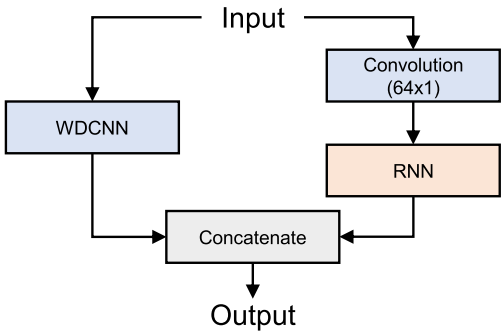


FIGURE 8. RNN-WDCNN architecture.

C. OPTIMIZATION METHODS

We used four optimizers: SGD, Momentum [37], RMSProp [38], and Adam [39] in this benchmarking study. They are the most popular optimizers for deep learning model training, which update parameters using a gradient. Before describing the parameter update rule for each optimizer, we define the model parameters, softmax output of the model, loss function, batch size, and i^{th}input as θ, $f(\cdot;\theta)$, $L(\cdot)$, n, (x_i, y_i), respectively. Algorithms 1–4 describe the update rules of the optimizers.

SGD is the simplest deep learning optimization method that updates parameters in the direction where the loss function decreases the fastest, that is, in the opposite direction of the gradient. This method requires only one crucial hyperparameter, the learning rate, which determines the step size of the parameter update. However, a fixed learning rate may lead to inefficiencies in the model training. For example, a low learning rate can easily reduce the training speed. However, the models often fail to converge to the optimal loss under a high learning rate.

Algorithm 1 SGD

Require: learning rate $\eta > 0$

1: **repeat**
2: $\quad \mathbf{g} = \frac{1}{n}\nabla_\theta \sum_i L(f(x_i;\theta), y_i)$
3: $\quad \theta = \theta - \eta\mathbf{g}$
4: **until** Converge

Momentum [37] was proposed to accelerate the training speed of SGD. It introduced the information of past gradients v to update the parameter, as described in Algorithm 2. In this algorithm, the momentum factor γ determines the contributions of the previous gradient. The larger the value of γ, the more the previous gradient affects the training. In general, frequently selected values for the momentum factor are 0.5, 0.9, and 0.99.

The learning rate is the most significant hyperparameter of optimizers; however, tuning the learning rate for a specific task is difficult. In recent studies, algorithms that adaptively change the learning rates of parameters, such as RMSProp and Adam, have been introduced to address this problem.

RMSProp uses an exponentially weighted moving average of the gradient to adapt to the learning rate. This method

Algorithm 2 Momentum

Require: learning rate $\eta > 0$, momentum factor $0 \leq \gamma < 1$

$v = 0$
repeat
  $\mathbf{g} = \frac{1}{n}\nabla_\theta \sum_i L(f(x_i;\theta), y_i)$
  $v = \gamma v - \eta \mathbf{g}$
  $\theta = \theta + v$
until Converge

modifies Adagrad and performs better in non-convex environments. RMSProp has four hyperparameters: η, α, γ, and ϵ. η is the initial learning rate, and α is the smoothing factor of the exponentially weighted moving average. γ is the momentum factor, which is the same as that in the Momentum algorithm. ϵ is a numerical constant that prevents division error by zero.

Recently, Adam [39] has been the most popular adaptive learning rate algorithm. It uses the first- and second-order moments of the gradient to change the learning rate adaptively. Adam may be regarded as a combination of RMSProp and Momentum; however, a significant distinction is that the moments of the gradient are rescaled. Adam has four hyperparameters: η, β_1, β_2, and ϵ, which indicate the initial learning rate, smoothing factors of the first- and second-order moments, and numerical constant, respectively. The commonly used default values for the hyperparameters are $\eta = 0.001$, $(\beta_1, \beta_2) = (0.9, 0.999)$, and $\epsilon = 10^{-8}$.

Algorithm 3 RMSProp

Require: learning rate $\eta > 0$, smoothing factor $0 \leq \alpha < 1$, momentum factor $0 \leq \gamma < 1$, numerical constant ϵ

$v = 0, m = 0$
repeat
  $\mathbf{g} = \frac{1}{n}\nabla_\theta \sum_i L(f(x_i;\theta), y_i)$
  $v = \alpha v + (1-\alpha)\mathbf{g} \bigotimes \mathbf{g}$
  $m = \gamma m + \frac{\eta}{\sqrt{v}+\epsilon}\mathbf{g}$
  $\theta = \theta - m$
until Converge

D. MODEL TRAINING AND HYPERPARAMETER TUNING

As described in the previous sections, we performed a two-stage benchmarking test for $2 \times 7 \times 4 = 56$ different configuration options using two datasets, seven models, and four optimizers. In the first stage, we trained the models 64 times for each optimizer and batch size in the initial search space of hyperparameters. Next, we refined the hyperparameter search space based on these results and trained the models 16 times again for the refined search space in the second stage. Finally, we evaluated the noise-robustness and the impacts of optimizers and batch size on the model performance.

The initial hyperparameter search space was set based on a study of large-scale optimizer hyperparameter tuning conducted in [36]. Table 4 summarizes the initial search space. Similar to most previous studies on hyperparameter

Algorithm 4 Adam

Require: learning rate $\eta > 0$, smoothing factor $0 \leq \beta_1, \beta_2 < 1$, numerical constant ϵ

$v = 0, m = 0, t = 0$
repeat
  $\mathbf{g} = \frac{1}{n}\nabla_\theta \sum_i L(f(x_i;\theta), y_i)$
  $m = \beta_1 m + (1-\beta_1)\mathbf{g}$
  $v = \beta_2 v + (1-\beta_2)\mathbf{g} \bigotimes \mathbf{g}$
  $\hat{m} = \frac{m}{1-\beta_1^t}$
  $\hat{v} = \frac{v}{1-\beta_2^t}$
  $\Delta\theta = -\eta \frac{\hat{m}}{\sqrt{\hat{v}}+\epsilon}$
  $\theta = \theta + \Delta\theta$
  $t = t + 1$
until Converge

tuning, we sampled the hyperparameters γ of Momentum, (α, γ) of RMSProp, and (β_1, β_2) of Adam from $(1-\gamma)$, $(1-\alpha, 1-\gamma)$, and $(1-\beta_1, 1-\beta_2)$, respectively, following a quasi-random uniform distribution. Because hyperparameters have multiplicative effects on updating the parameters [40], [41], we sampled the hyperparameter set from a log scale. For example, when the search space of the learning rate is $[10^{-4}, 10^{0}]$, the hyperparameters are sampled from $10^{\mathcal{U}[-4,0]}$, where $\mathcal{U}$ denotes a uniform distribution.

Pseudo-random sampling can generate a biased distribution of hyperparameters. Therefore, we used a quasi-random distribution to sample the hyperparameters [53]. Fig. 9 shows the difference between the pseudo-random and quasi-random sampling for two variables in the range $[10^{-2}, 10^{0}]$. The sampling results showed that the samples from the quasi-random distribution had a lower discrepancy than those from the pseudo-random distribution.

We trained each model for 100 epochs for each sampled hyperparameter set and selected the result with the lowest validation loss. Cross-entropy was used as the cost function L given by (1). We let C, $y_i^{(j)}$, and $f(x_i;\theta)^{(j)}$ be the number of class, one-hot encoded label for j^{th} class of ith data, and softmax output for j^{th} class of ith data, respectively.

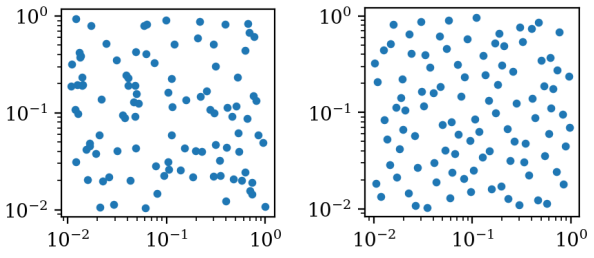
$$L(f(x_i;\theta), y_i) = -\sum_{j}^{C} y_i^{(j)} \log f(x_i;\theta)^{(j)} \tag{1}$$

After model training, we evaluated the models using both the test accuracy of the noise-free and noisy data. The test accuracy of the noise-free data represents the general performance of the model. By contrast, we used data with additive white Gaussian noise, which is the most frequently employed method for evaluating the noise-robustness of models, to investigate the model performance in a harsh industrial environment often exposed to noise interference.

Additive white Gaussian noise was generated as follows. First, we calculated the signal power using (2), where N and x_i denote the length and i^{th} element of the signal, respectively. The power of the noise was then derived from

TABLE 4. Initial hyperparameter search space.

SGD		Momentum		RMSProp		Adam	
Hyperparameter	Search Space	Hyperparameter	Search Space	Hyperparameter	Search Space	Hyperparameter	Search Space
η	$[10^{-4}, 10^{0}]$	η	$[10^{-4}, 10^{0}]$	η	$[10^{-4}, 10^{-1}]$	η	$[10^{-4}, 10^{-1}]$
		$1-\gamma$	$[10^{-3}, 10^{0}]$	$1-\alpha$	$[10^{-3}, 10^{0}]$	$1-\beta_1$	$[10^{-3}, 10^{0}]$
				$1-\gamma$	$[10^{-3}, 10^{0}]$	$1-\beta_2$	$[10^{-4}, 10^{-1}]$
				ϵ	$[10^{-10}, 10^{0}]$	ϵ	$[10^{-10}, 10^{0}]$



(a) Pseudo-random sampling (b) Quasi-random sampling

FIGURE 9. Comparison between pseudo-random and quasi-random sampling.

(3) with a specific signal-to-noise ratio (SNR), denoted as SNR_{dB}. Finally, Gaussian noise from the normal distribution $N(0, \sqrt{P_{\text{noise}}})$ was added to the original signal. To evaluate the robustness of the models against noisy data, we used the average accuracy using the noisy data with SNR $[-4, -2, 0, 2, 4]_{\text{dB}}$.

$$P_{\text{signal}} = \frac{1}{N}\sum_{i}^{N} x_i^2 \tag{2}$$

$$\text{SNR}_{\text{dB}} = 10\log_{10}\frac{P_{\text{signal}}}{P_{\text{noise}}} \tag{3}$$

IV. EXPERIMENTAL RESULTS AND DISCUSSION

This section presents the benchmarking results of a two-stage hyperparameter tuning for bearing fault diagnosis using various deep learning models and discusses the impact of the tuning results in both noise-free and noisy environments on model performance. To this end, we first tuned the hyperparameters in the initial search space, as presented in Table 4. Subsequently, we refined the search space and trained the models again based on the refined search space.

Table 5 summarizes the experimental setup. We used PyTorch [54] and PyTorch Lightning [55] to implement and train the models on two NVIDIA GeForce RTX2080 SUPER GPUs in parallel. For reproducibility, we also fixed a random seed in all the experiments. Our source code used for the experiment is available at: https://github.com/junior209lsj/FaultDiagnosisOptimizerBenchmark.

A. CASE 1: CWRU DATASET

In the training results of the first stage in the CWRU dataset, we observed that the smoothing factors and the numerical constant ϵ of the adaptive learning rate methods, that is, RMSProp and Adam, had only minor effects on the model performance. Therefore, we focused on the impacts of the learning rate and momentum factor.

Figs. 10–15 show the relationship between the hyperparameter and model performance on the CWRU dataset with the optimizers and varying batch sizes through scatter plots and trend lines. In the figures, each point represents the pair of test accuracy and hyperparameter for a training result. Trend lines for each batch size were also plotted using locally weighted scatterplot smoothing to illustrate how the accuracy of the model changed with different hyperparameter values. The results showed that the model performances in both noise-free and noisy environments were significantly affected by the learning rate η for all optimizers and the momentum factor γ for Momentum and RMSProp, which were the dominant hyperparameters for determining the training speed of the models. For example, when η was high, the step size of the parameter update increased. High momentum factor γ also accelerated the model training.

TABLE 5. Hardware and software specifications for the experiment.

Type	Specification
OS	Ubuntu 18.04
CPU	Intel® Core™ i9-10900K @ 3.70GHz
RAM	128GB
GPU	NVIDIA GeForce RTX 2080 SUPER x2
CUDA version	11.2
CUDNN version	7.6.5
pyTorch version	1.12.1
pyTorch Lightning version	1.7.7

As shown in Fig. 10, in SGD, the learning rate significantly affected the performance of all models, except for RNN-WDCNN. For instance, in WDCNN and TICNN, when the learning rate was lower than 10^{-3}, the performance decreased in noise-free and noisy environments. DCN and SRDCNN showed patterns similar to those of WDCNN and TICNN in a noise-free environment. However, when the learning rate was higher than 10^{-1}, the performance of SRDCNN rapidly decreased in both environments, and the performance of DCN also decreased slightly in the noisy environment.

The results for the 2D CNN models, STIM-CNN and STFT-CNN, showed slightly different patterns compared with the 1D CNN models. The models had clear ranges of learning rates, producing the best model performance. In a noise-free environment, STIM-CNN and STFT-CNN achieved the maximum performance at a learning rate of around 10^{-2}. As for the noisy environment, the performance of STFT-CNN was the best at a learning rate of around 10^{-1}; however, the best learning rate for STIM-CNN varied according to the batch size. Among all candidate models, RNN-WDCNN was the least affected by the learning rate. In a noise-free environment, the accuracy of RNN-WDCNN was approximately 100%, and the higher the learning rate, the higher the accuracy in a noisy environment.

Figs. 11 and 12 show the effects of the learning rate and momentum factor, respectively, on the model performance using the Momentum optimizer. We observed that as the

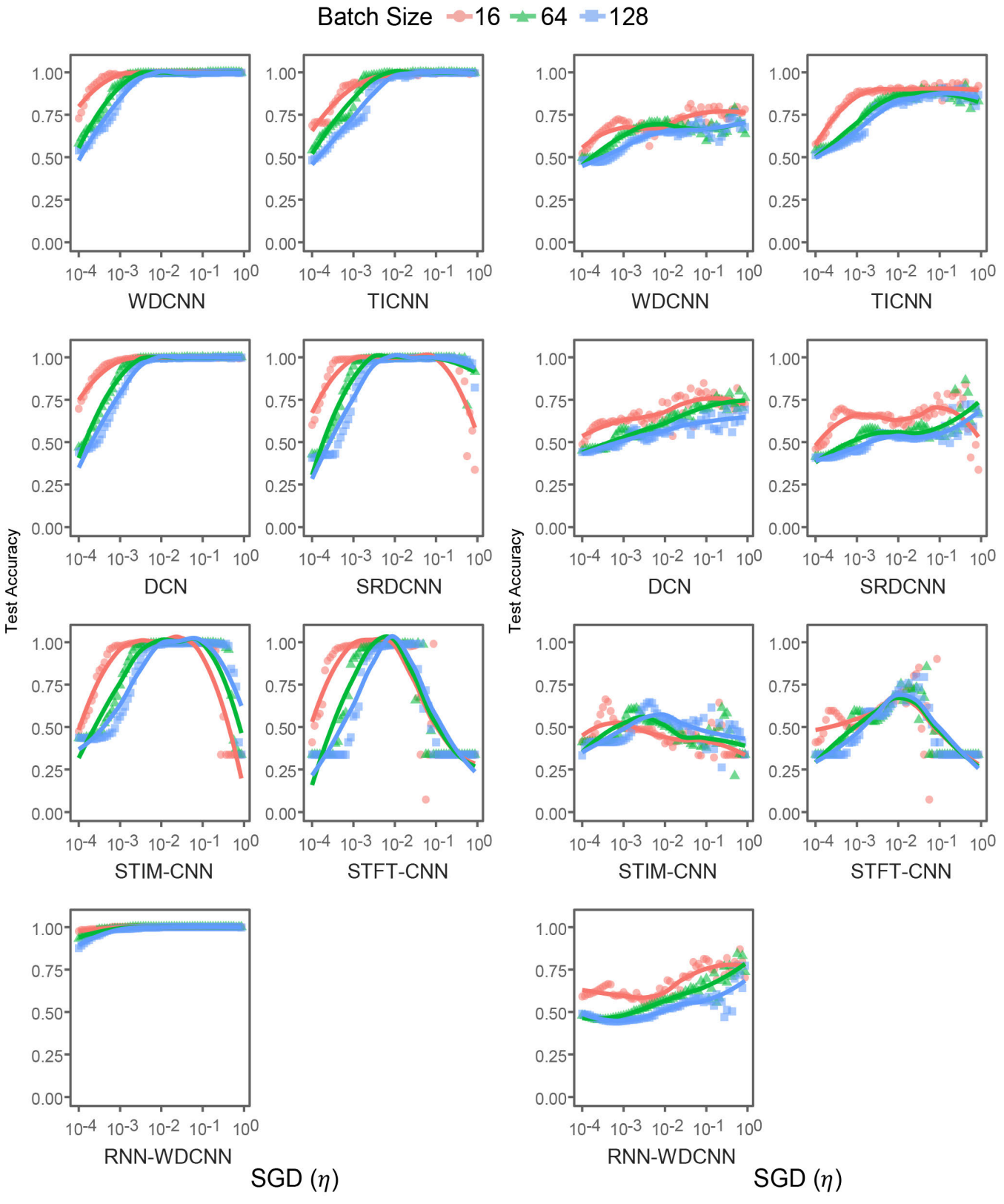
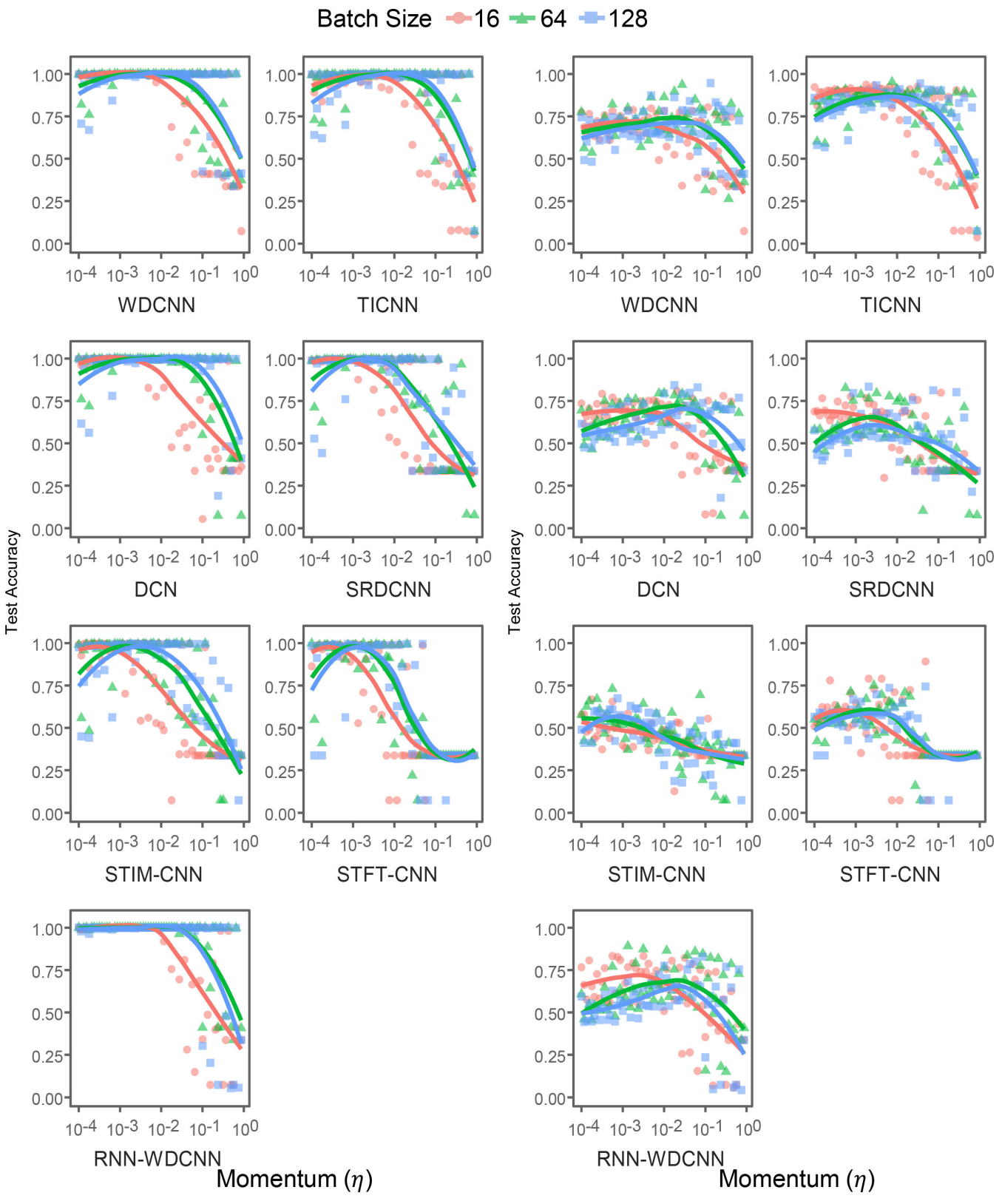


FIGURE 10. Hyperparameter tuning using the CWRU dataset and SGD optimizer (learning rate).



(a) Dataset Without Noise

(b) Dataset With Noise

FIGURE 11. Hyperparameter tuning using the CWRU dataset and Momentum optimizer (learning rate).

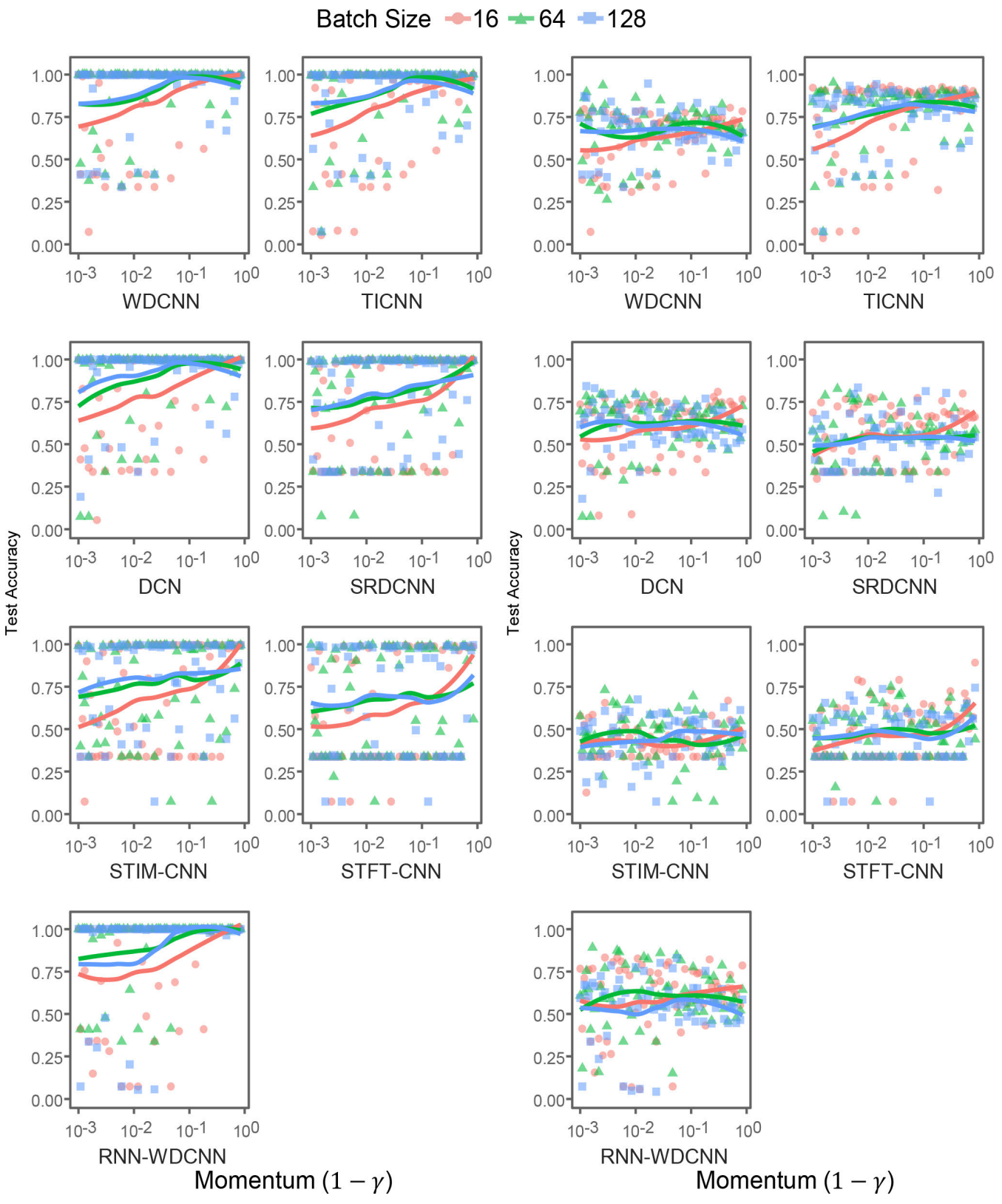


FIGURE 12. Hyperparameter tuning using the CWRU dataset and Momentum optimizer (momentum factor).

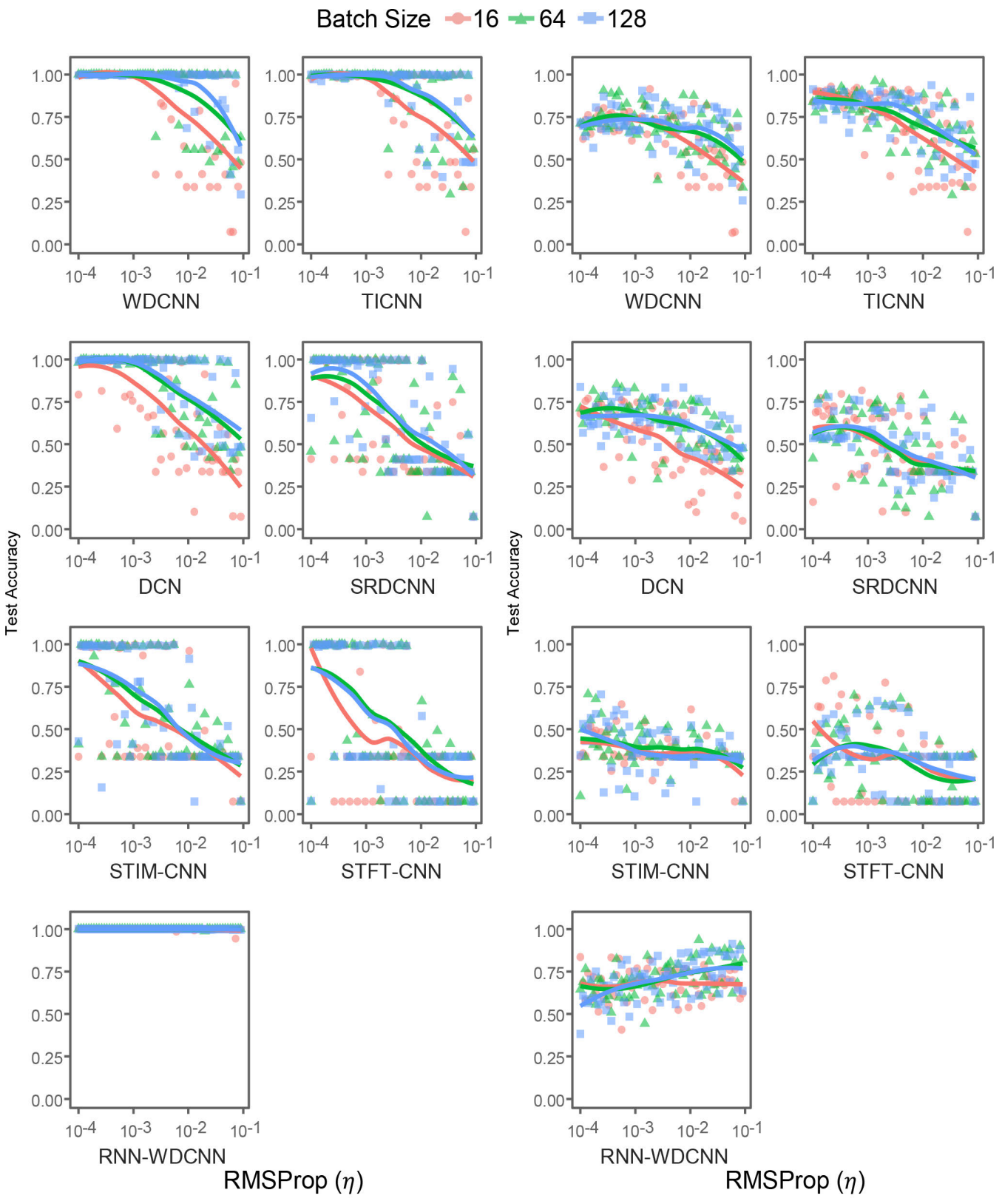


FIGURE 13. Hyperparameter tuning using the CWRU dataset and RMSProp optimizer (learning rate).

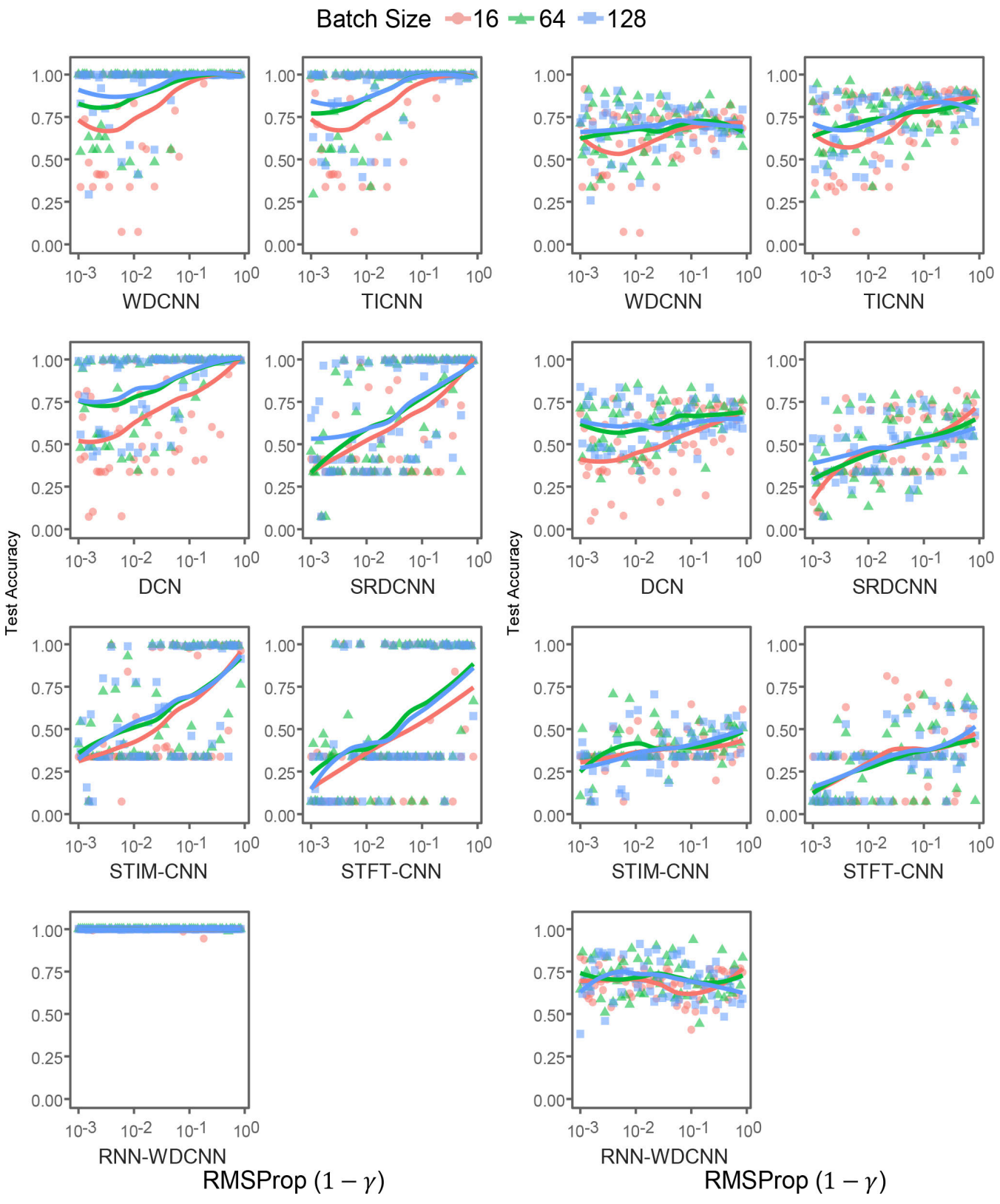


FIGURE 14. Hyperparameter tuning using the CWRU dataset and RMSProp optimizer (momentum factor).

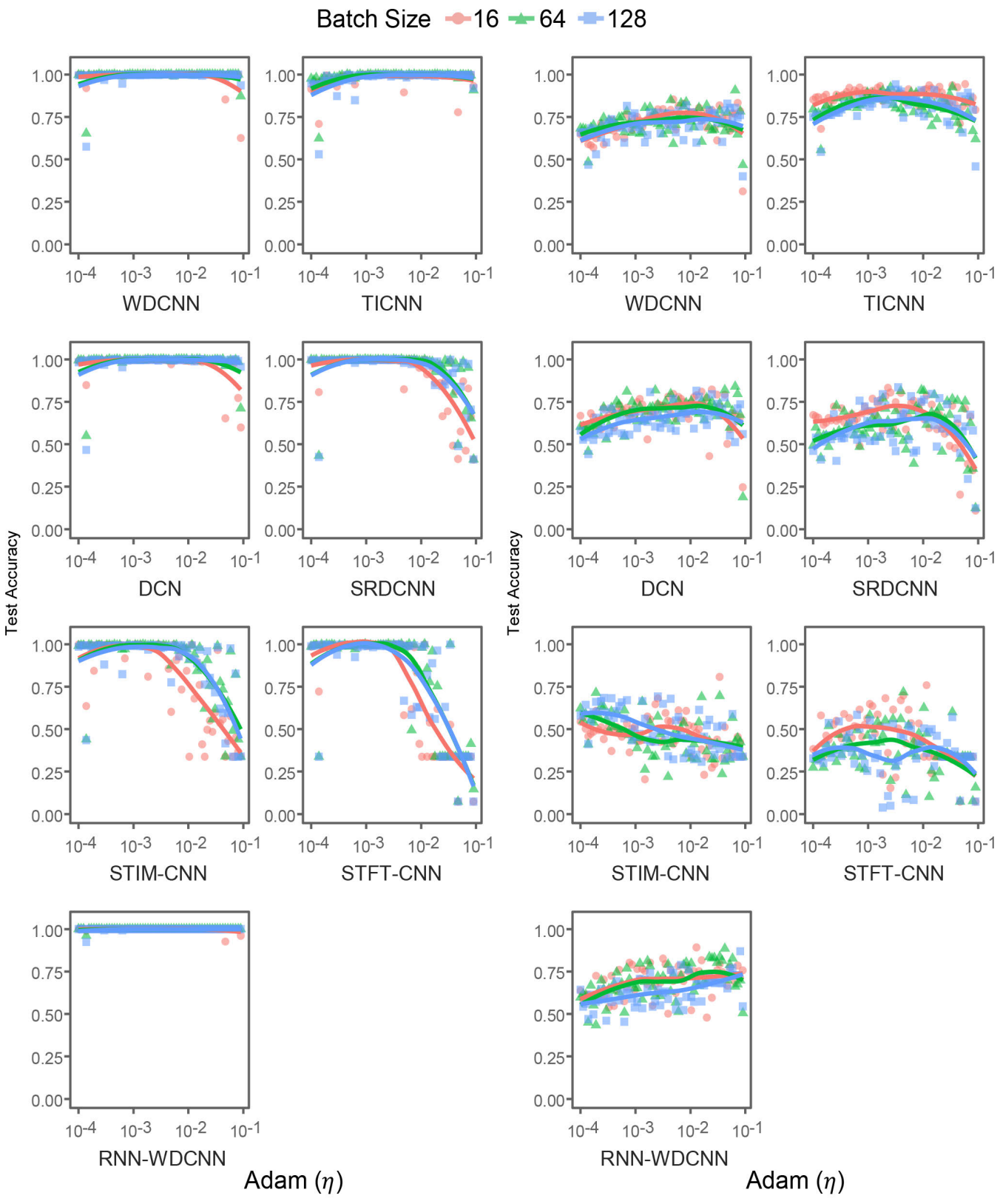


FIGURE 15. Hyperparameter tuning using the CWRU dataset and Adam optimizer (learning rate).

TABLE 6. Refined hyperparameter search space for the CWRU dataset.

Model	SGD	Momentum		RMSProp ($\alpha = 0.99, \epsilon = 10^{-8}$)		Adam ($\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$)
	η	η	$1-\gamma$	η	$1-\gamma$	η
WDCNN	$[10^{-2}, 10^{0}]$	$[10^{-3}, 10^{-1}]$	$[10^{-1}, 10^{0}]$	$[10^{-4}, 10^{-2}]$	$[10^{-1}, 10^{0}]$	$[10^{-4}, 10^{-2}]$
TICNN	$[10^{-2}, 10^{0}]$	$[10^{-3}, 10^{-1}]$	$[10^{-1}, 10^{0}]$	$[10^{-4}, 10^{-2}]$	$[10^{-1}, 10^{0}]$	$[10^{-4}, 10^{-2}]$
DCN	$[10^{-2}, 10^{0}]$	$[10^{-3}, 10^{-1}]$	$[10^{-1}, 10^{0}]$	$[10^{-4}, 10^{-2}]$	$[10^{-1}, 10^{0}]$	$[10^{-4}, 10^{-2}]$
SRDCNN	$[10^{-2}, 10^{0}]$	$[10^{-3}, 10^{-2}]$	$[10^{-1}, 10^{0}]$	$[10^{-4}, 10^{-3}]$	$[10^{-1}, 10^{0}]$	$[10^{-3}, 10^{-2}]$
STIM-CNN	$[10^{-3}, 10^{-2}]$	$[10^{-3}, 10^{-2}]$	$[10^{-1}, 10^{0}]$	$[10^{-4}, 10^{-3}]$	$[10^{-1}, 10^{0}]$	$[10^{-4}, 10^{-3}]$
STFT-CNN	$[10^{-3}, 10^{-1}]$	$[10^{-3}, 10^{-2}]$	$[10^{-1}, 10^{0}]$	$[10^{-4}, 10^{-3}]$	$[10^{-1}, 10^{0}]$	$[10^{-4}, 10^{-2}]$
RNN-WDCNN	$[10^{-2}, 10^{0}]$	$[10^{-4}, 10^{-2}]$	$[10^{-1}, 10^{0}]$	$[10^{-4}, 10^{-2}]$	$[10^{-1}, 10^{0}]$	$[10^{-4}, 10^{-1}]$

learning rate increased, the model performance declined more rapidly than SGD in noise-free and noisy environments when using the Momentum optimizer. For example, the performances of all candidate models in both environments decreased at learning rates higher than 10^{-2}. Moreover, with a higher momentum factor, that is, approximately 1, models trained in a small batch size generally exhibited poor test accuracy. This was because the Momentum optimizer trained models faster than SGD by employing previous gradients.

In most cases, when the learning rate was high, a small batch size led to poor model performances. Conversely, at lower learning rates, the model performance increased when the batch size was small. However, in 2D CNN models, the correlations between the learning rate and batch size on the model performance were unclear because their overall performances were low in a noisy environment.

Based on the results illustrated in Fig. 13, we can observe that the model performance degradation followed by increasing the learning rate was the most outstanding in the RMSProp optimizer. Similar to the Momentum optimizer, a high learning rate or momentum factor reduced the model performance in both environments, as shown in Figs. 13 and 14. Furthermore, the performance reduction followed by the learning rate increment when using the RMSProp optimizer was higher than that of using the Momentum optimizer.

However, unlike the Momentum case, RNN-WDCNN exhibited consistent performance in all hyperparameter ranges. RNN-WDCNN maintained approximately 100% accuracy when using the noise-free data and exhibited approximately 70–75% average accuracy when using the noisy data in all hyperparameter ranges. Although some studies have shown the effectiveness of the RMSProp optimizer for training small RNN-based models [56], [57], [58], research on the effect of RMSProp learning rate is still limited. In our experiments, we found that the performance of RNN-WDCNN was less sensitive to learning rate variation.

Fig. 15 illustrates the performances of the models under various learning rates using the Adam optimizer. In the Adam case, the performances of DCN, SRDCNN, STIM-CNN, and STFT-CNN declined when the learning rate approached 10^{-1}. In addition, the performance was worse for a small batch size when the learning rate was approximately 10^{-1}. However, among the optimizers, the Adam optimizer was the least sensitive to variations in hyperparameters because WDCNN, TICNN, and RNN-WDCNN maintained their performances regardless of the changes in the learning rate, in most cases.

The hyperparameter tuning results in the first stage show that the hyperparameters had significant impacts on the model performance. In particular, the learning rate and momentum factor, which determine the model training speed, significantly affected the model accuracy in noise-free and noisy environments. In general, the model performance decreased when the model training speed was high for small batch sizes or when the model training speed was low for large batch sizes. Based on this result, we refined the hyperparameter search spaces, including those which did not result in model performance reduction. Because the hyperparameters α and ϵ of RMSProp and β_1, β_2, and ϵ of Adam had trivial effects on the model performance, we set them to the default values. In most cases, the model performance decreased rapidly when the momentum factor exceeded 0.9. Therefore, we used $10^{-1} \leq 1-\gamma \leq 10^{0}$ for the new search space. Table 6 summarizes the refined search space for the hyperparameters.

We trained each model 16 times using the refined search space in the second stage and compared the model performance for different optimizers and batch sizes. Because the average test accuracies of all the models in a noise-free environment were higher than 95%, only the results for the noisy environment were further analyzed.

Fig. 16 shows the model training results in the refined search space for the noisy data. The value in the bar chart is the average test accuracy of the 16 results in the noisy data, and the error bar shows the 95% confidence interval. The results indicated that no optimizer achieved the best performance across all the models. However, it can be observed that the batch size and noise-robustness correlate, except for STIM-CNN. A small batch size led to high model performance; however, the relationship between batch size and model performance was not clear in STIM-CNN. 2D CNNs were less stable and had lower accuracies than 1D CNNs in noisy environments. In particular, among all candidate models, TICNN achieved the highest and most stable performance.

B. CASE 2: MFPT DATASET

We conducted the same experiment as described in Section IV-A using the MFPT dataset. The results also show that smoothing factors, namely, α, β_1, β_2, and the numerical constant ϵ had trivial impacts on model performance. Thus, similar to the first stage experiment using the CWRU dataset, we investigated the effects of the learning rate and momentum factor on the model performance in detail.

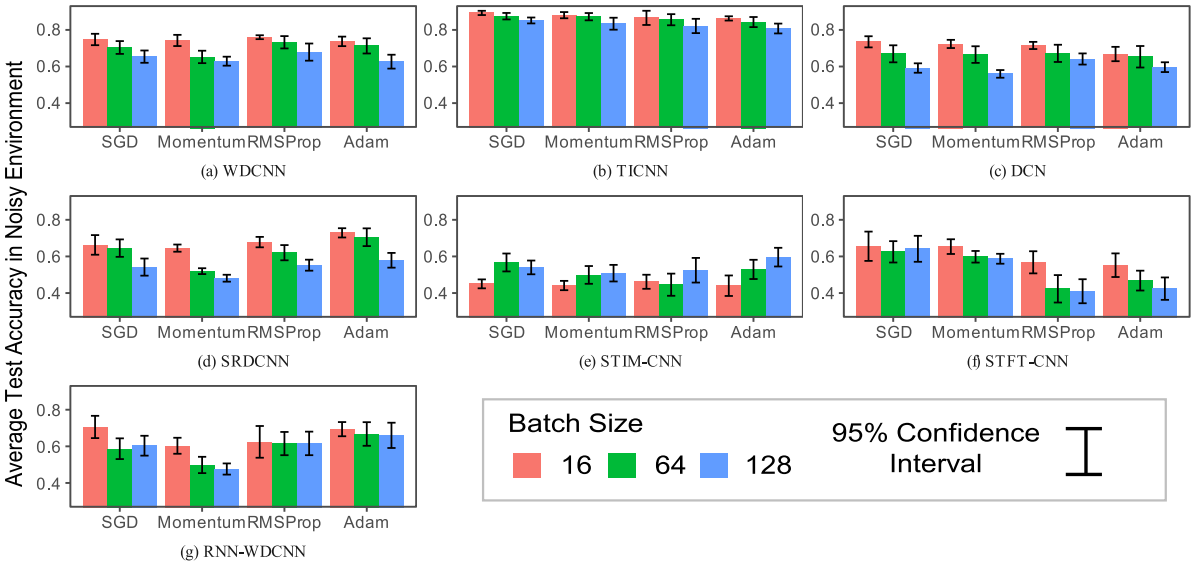


FIGURE 16. Model performance in a noisy environment with refined search space using the CWRU dataset.

Figs. 17–22 show the hyperparameter tuning results for the first stage. In the initial search space, the parameters η for all optimizers and γ for Momentum and RMSProp optimizers significantly affected the model performance. This result shows that the learning rate and momentum factor were critical hyperparameters in both datasets.

Fig. 17 shows the relationship between the learning rate and the model performance using SGD. Similar to the CWRU case in Fig. 10, each model had a specific learning rate range to produce high model performance. The performances of WDCNN and DCN rapidly decreased at learning rates lower than 10^{-3}. The performance of TICNN decreased when the learning rates were lower than 10^{-2}. By contrast, 2D CNNs maintained their performance at learning rates between 10^{-3} and 10^{-1} when using noise-free data. For all models, at high learning rates, training with large batch sizes resulted in a higher accuracy than training with small batch sizes. By contrast, a small batch size led to a higher model performance at a low learning rate.

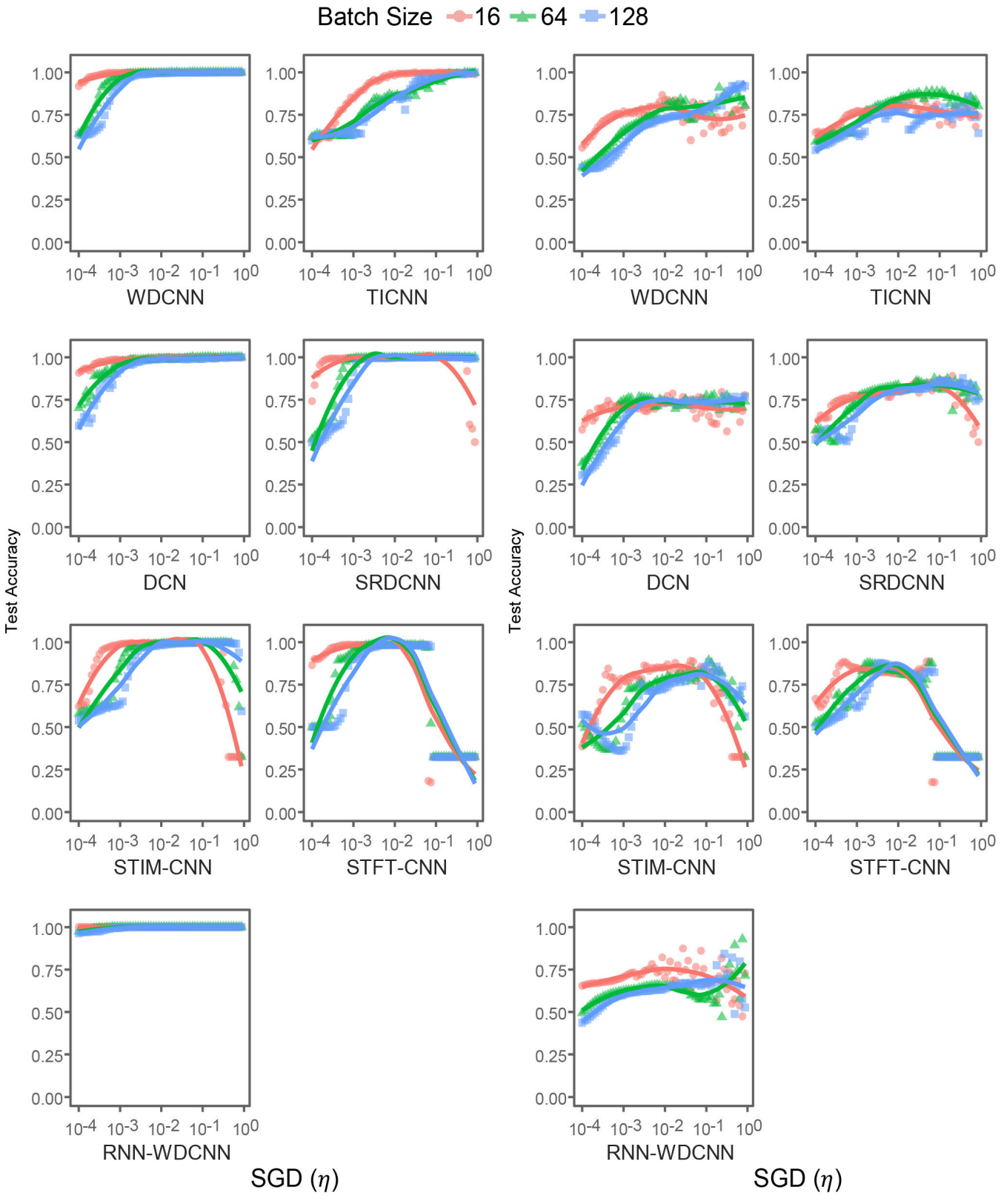
In both datasets, RNN-WDCNN exhibited the highest performance in the entire learning rate range in a noise-free environment and a learning rate of approximately 1 in a noisy environment. However, the performance of TICNN was more sensitive to the learning rate when using the MFPT dataset than when using the CWRU dataset. TICNN achieved the highest test accuracy when the learning rate was higher than 10^{-2} in noise-free and noisy environments using the CWRU dataset. However, for the MFPT dataset, the highest test accuracy was achieved when the learning rate was approximately 1 in a noise-free environment. The other models had similar tendencies for both datasets.

The hyperparameter tuning results showed similar patterns in the two datasets using the Momentum optimizer. As shown in Fig. 18, the test accuracies of all the models decreased rapidly when the learning rates were higher than 10^{-2}. At high learning rates, training results with small batch sizes generally had lower accuracies than those with large batch sizes. In particular, the 2D CNN models, STIM-CNN and STFT-CNN, exhibited a noticeable performance reduction at high learning rates compared to the other models. In addition, when the momentum factor was approximately 1, the performances of the models decreased in most cases, as shown in Fig. 19. However, the performance reduction owing to the increasing momentum factor was smaller than that in the learning rate case.

Similar to Momentum, the performances of the models decreased when the learning rate increased in RMSProp, as shown in Fig. 20. However, the degree of performance reduction caused by the increase in learning rate was smaller for the MFPT dataset than for the CWRU dataset. In addition, whereas the performances of WDCNN, TICNN, and DCN exhibited little variation based on the learning rate in a noisy environment, the performances of SRDCNN, STIM-CNN, and STFT-CNN were more sensitive to the learning rate. Similar to the experiment using the CWRU dataset, the performance of RNN-WDCNN was stable at all learning rates in both noise-free and noisy environments.

Fig. 21 shows the impact of the momentum factor on the model performance. When the momentum factor was approximately 1, the model performance decreased; however, the effect of the momentum factor was smaller than that of the learning rate.

Fig. 22 shows the relationship between the learning rate of the Adam optimizer and model performance on the MFPT dataset. The results showed that, among the four optimizers, the learning rate of Adam had the least impact on the model performance. However, for SRDCNN, STIM-CNN, and STFT-CNN, when the learning rates were higher than 10^{-2}, the performance decreased regardless of the existence of noise. In addition, the model performance of TICNN decreased when the learning rate was less than 10^{-3}. Similar to the other optimizers, at higher learning rates,



(a) Dataset Without Noise

(b) Dataset With Noise

FIGURE 17. Hyperparameter tuning using the MFPT dataset and SGD optimizer (learning rate).

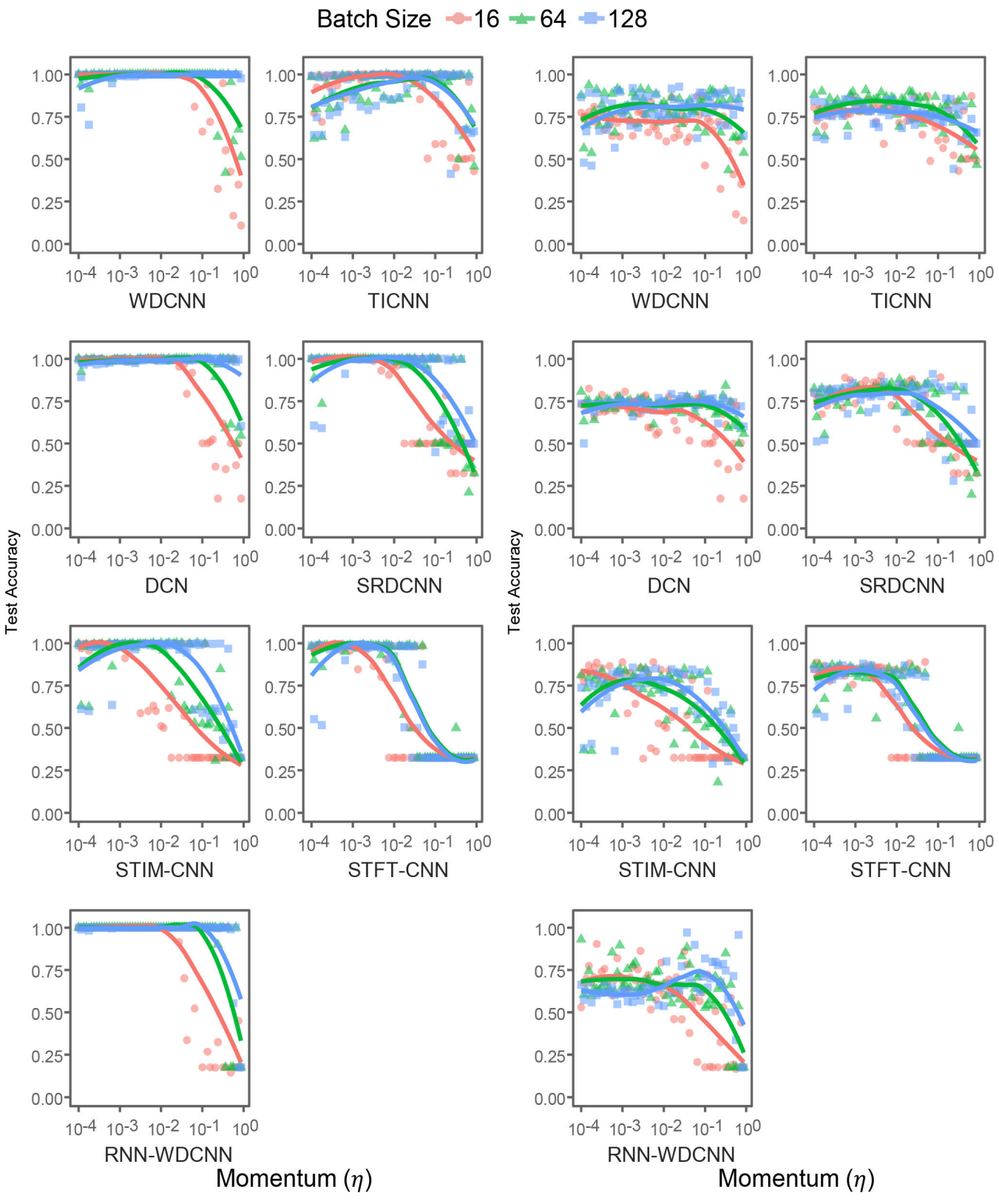


FIGURE 18. Hyperparameter tuning using the MFPT dataset and Momentum optimizer (learning rate).

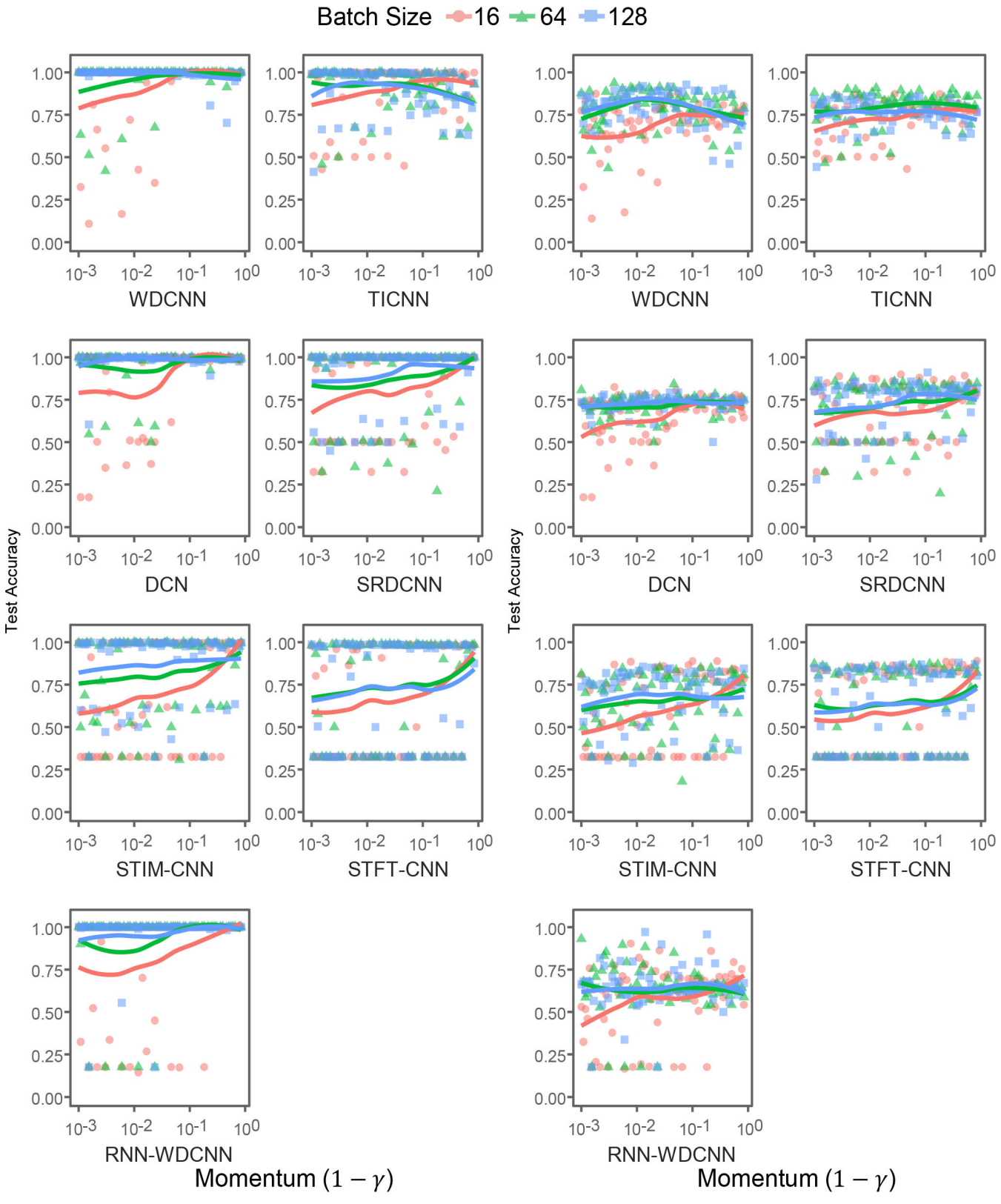


FIGURE 19. Hyperparameter tuning using the MFPT dataset and Momentum optimizer (momentum factor).

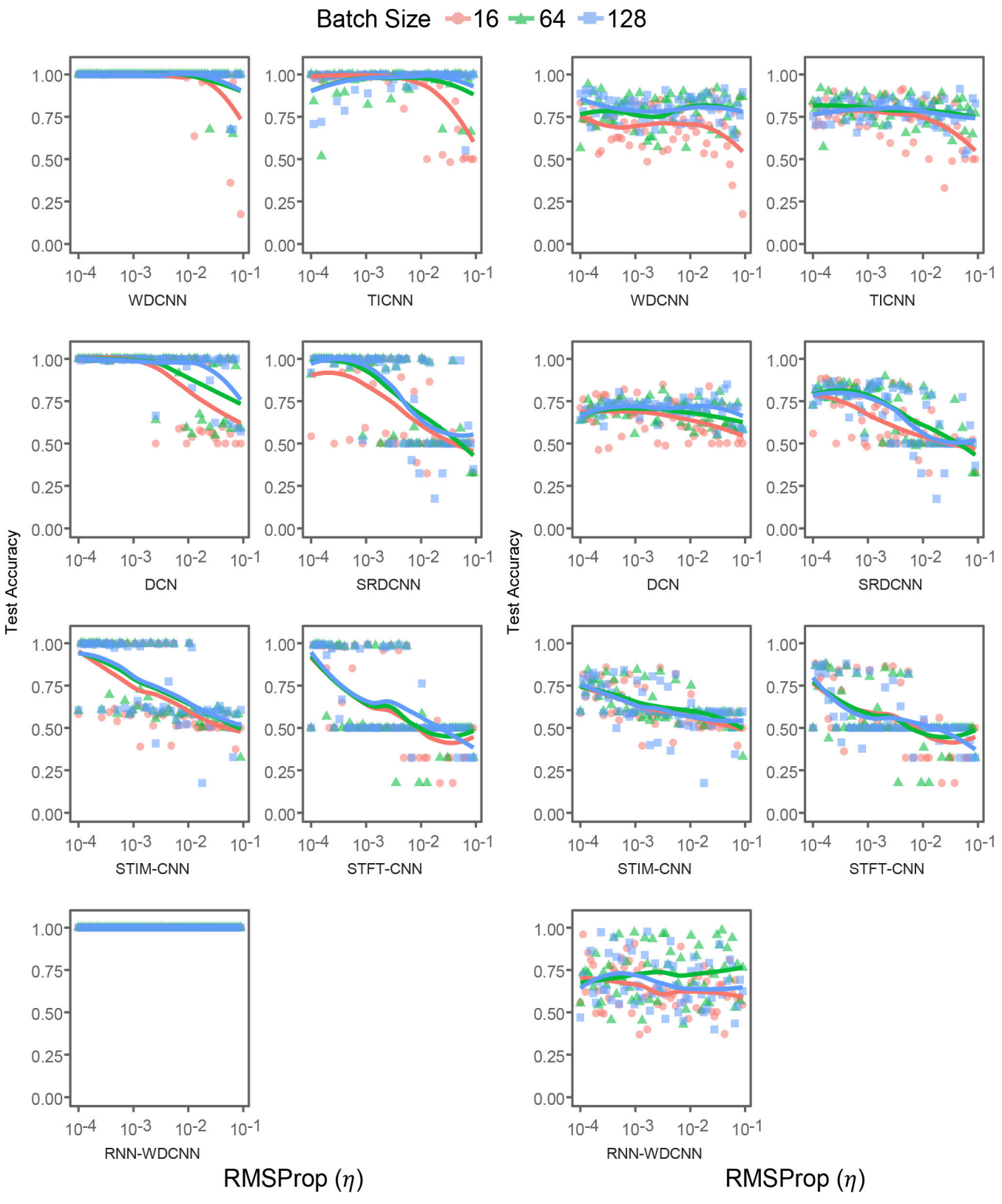


FIGURE 20. Hyperparameter tuning using the MFPT dataset and RMSProp optimizer (learning rate).

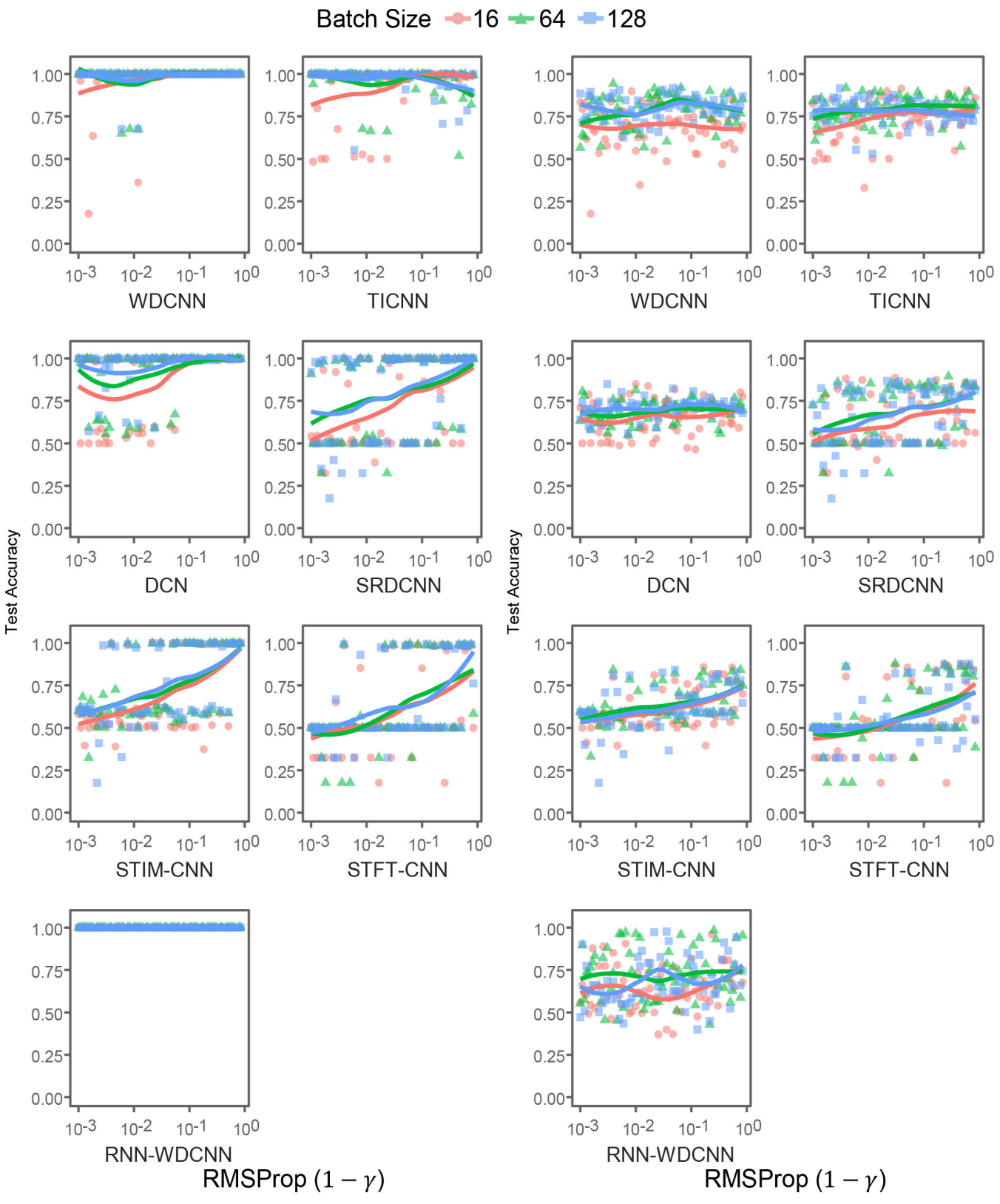


FIGURE 21. Hyperparameter tuning using the MFPT dataset and RMSProp optimizer (momentum factor).

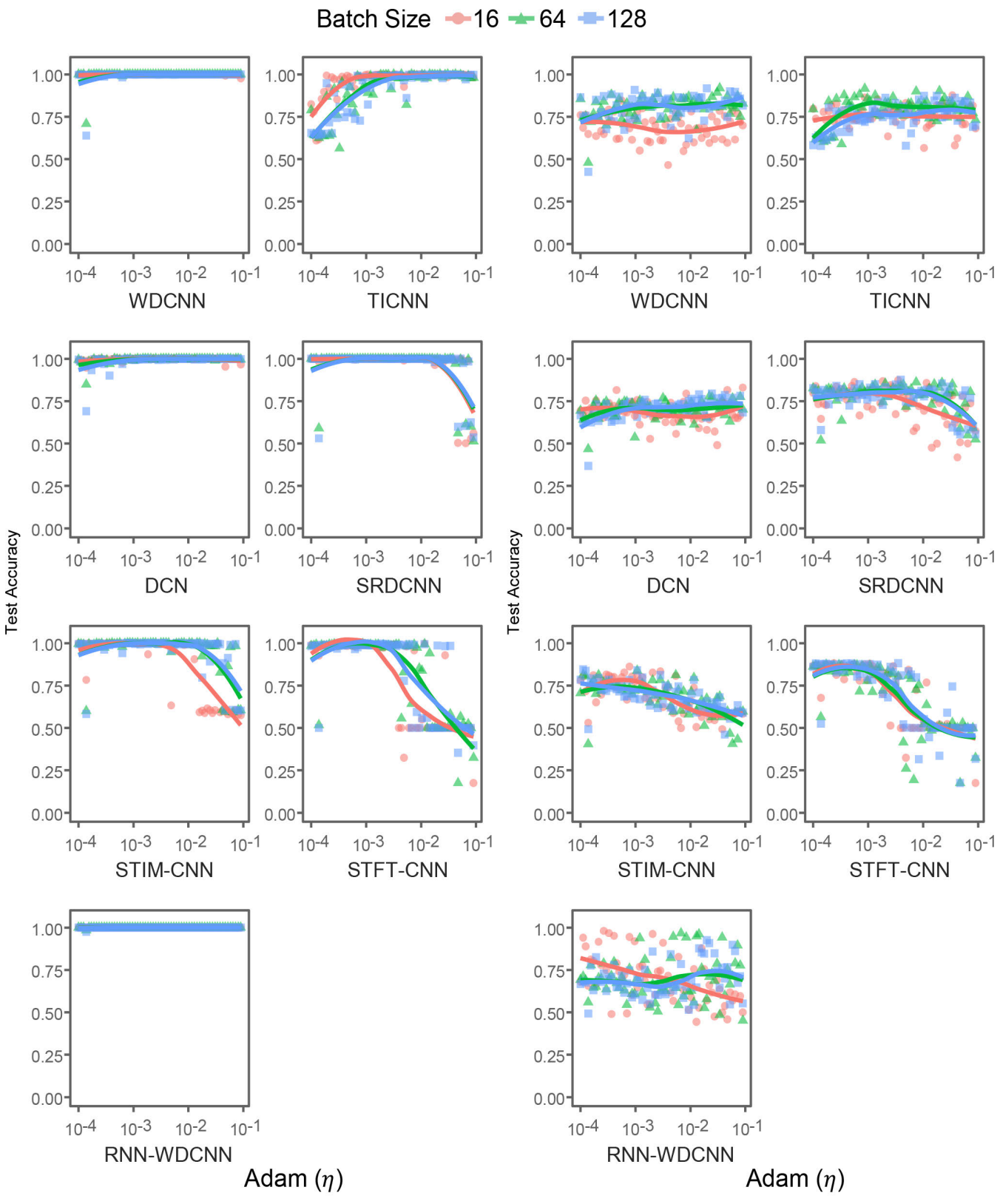


FIGURE 22. Hyperparameter tuning using the MFPT dataset and Adam optimizer (learning rate).

TABLE 7. Refined hyperparameter search space for the MFPT dataset.

Model	SGD	Momentum		RMSProp ($\alpha = 0.99, \epsilon = 10^{-8}$)		Adam ($\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$)
	η	η	$1-\gamma$	η	$1-\gamma$	η
WDCNN	$[10^{-2}, 10^{0}]$	$[10^{-3}, 10^{-1}]$	$[10^{-1}, 10^{0}]$	$[10^{-4}, 10^{-2}]$	$[10^{-1}, 10^{0}]$	$[10^{-4}, 10^{-2}]$
TICNN	$[10^{-2}, 10^{0}]$	$[10^{-3}, 10^{-1}]$	$[10^{-1}, 10^{0}]$	$[10^{-4}, 10^{-2}]$	$[10^{-1}, 10^{0}]$	$[10^{-3}, 10^{-1}]$
DCN	$[10^{-2}, 10^{0}]$	$[10^{-3}, 10^{-1}]$	$[10^{-1}, 10^{0}]$	$[10^{-4}, 10^{-2}]$	$[10^{-1}, 10^{0}]$	$[10^{-4}, 10^{-2}]$
SRDCNN	$[10^{-2}, 10^{0}]$	$[10^{-3}, 10^{-2}]$	$[10^{-1}, 10^{0}]$	$[10^{-4}, 10^{-3}]$	$[10^{-1}, 10^{0}]$	$[10^{-3}, 10^{-2}]$
STIM-CNN	$[10^{-3}, 10^{-1}]$	$[10^{-4}, 10^{-2}]$	$[10^{-2}, 10^{0}]$	$[10^{-4}, 10^{-3}]$	$[10^{-1}, 10^{0}]$	$[10^{-4}, 10^{-3}]$
STFT-CNN	$[10^{-3}, 10^{-1}]$	$[10^{-4}, 10^{-2}]$	$[10^{-2}, 10^{0}]$	$[10^{-4}, 10^{-3}]$	$[10^{-1}, 10^{0}]$	$[10^{-4}, 10^{-3}]$
RNN-WDCNN	$[10^{-2}, 10^{0}]$	$[10^{-4}, 10^{-2}]$	$[10^{-1}, 10^{0}]$	$[10^{-4}, 10^{-2}]$	$[10^{-1}, 10^{0}]$	$[10^{-4}, 10^{-1}]$

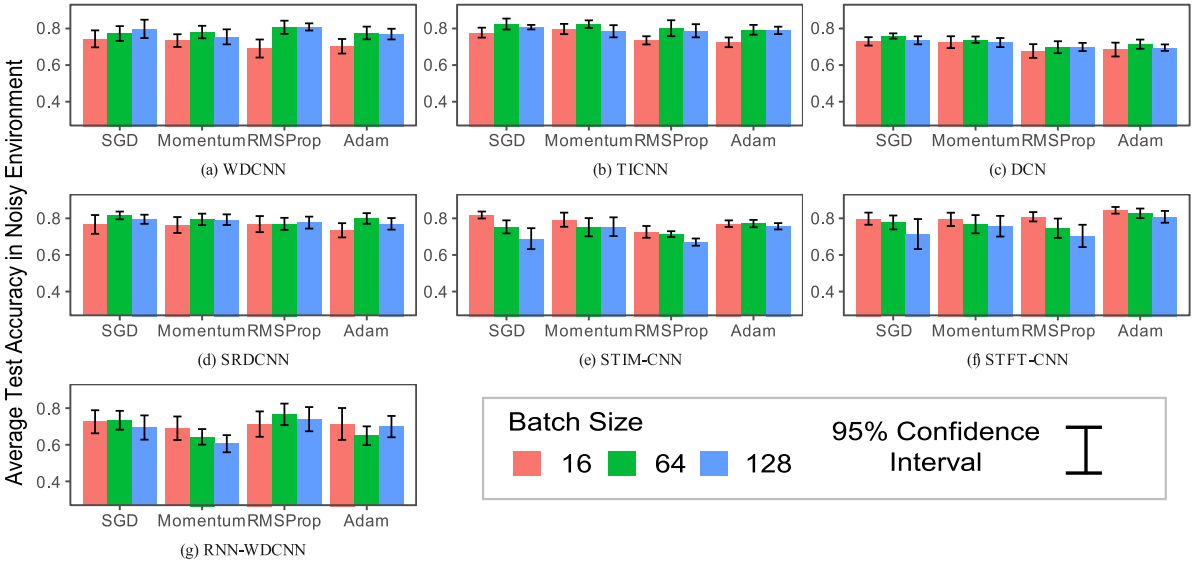


FIGURE 23. Model performance in a noisy environment with refined search space using the MFPT dataset.

TABLE 8. Comparison of candidate models in terms of accuracy, model size, and computation load.

Model	Accuracy(%) and 95% Confidence Interval				No. of Params (M)	MFLOPs
	CWRU (without noise)	CWRU (with noise)	MFPT (without noise)	MFPT (with noise)		
WDCNN	99.81±0.08	69.81±1.08	99.94±0.03	76.14±1.13	0.05	1.48
TICNN	99.10±0.36	85.51±0.75	97.13±0.84	78.72±0.84	0.07	2.63
DCN	99.53±0.10	65.72±1.15	99.72±0.07	71.50±0.77	0.16	9.17
SRDCNN	98.69±0.73	59.49±1.43	99.52±0.51	77.89±0.96	0.82	254.90
STIM-CNN	97.24±0.95	50.02±1.43	98.14±0.87	74.73±1.06	3.53	28.50
STFT-CNN	96.45±1.48	55.05±2.01	96.35±1.09	77.89±1.33	2.60	17.15
RNN-WDCNN	99.85±0.09	61.26±1.78	99.96±0.04	69.79±1.71	0.86	76.65

the models achieved a higher performance with larger batch sizes. By contrast, with smaller batch sizes, lower learning rates resulted in higher performances.

In the experimental results performed on the initial hyperparameter search space using the MFPT dataset, the overall relationship between the hyperparameters and model performance was similar for the two datasets. Furthermore, the model performance varied significantly depending on the hyperparameters. Therefore, based on the experimental results of the initial search space, we refined the hyperparameter search space using the same method used for the CWRU dataset. Table 7 summarizes the refined search space for the MFPT dataset.

For the refined search space, all models achieved a test accuracy higher than 95% in a noise-free environment; therefore, we analyzed in detail the performances of the models only for a noisy environment. Fig. 23 shows the results of the training repeated 16 times for each model using the MFPT dataset in the refined search space. Although decreasing the batch size was effective for noise-robustness using the CWRU dataset, only the 2D CNN models were more robust against noise when the batch size was small in the MFPT dataset. Except for TICNN, all models achieved higher test accuracies using the noisy data of the MFPT dataset than the CWRU dataset. TICNN was the most robust model against noise using the CWRU dataset but exhibited similar performance to WDCNN using the MFPT dataset. The performances of the 2D CNNs were also improved, which was similar to that of 1D CNNs.

C. MODEL PERFORMANCE COMPARISON

After two-stage training using the two datasets, we summarized the training results and evaluated the model performance, as shown in Table 8. The performance of the models was evaluated using the average test accuracies and their 95% confidence interval in both noise-free and noisy environments. The number of training results for each model

was $4 \times 3 \times 16 = 192$, representing all cases for the four optimizers and three batch sizes.

Moreover, on-device AI systems have gained attention in machine fault diagnosis for reducing the inference time and data traffic between edge devices on a factory floor and a server. Because on-device AI systems have limited hardware resources, we used the number of parameters and the million floating point operations per second (MFLOPs) as the evaluation metrics of the candidate models.

Overall, the 1D CNN models outperformed the 2D CNNs in noise-free and noisy environments, particularly for the CWRU dataset with noise. Furthermore, the number of parameters of the 1D CNN models was smaller than that of the 2D CNN models, indicating that 1D CNN models require smaller memory storage than 2D CNN models. In a noisy environment, the performances of the models, except for TICNN, were notably higher for the MFPT dataset than for the CWRU dataset. TICNN was the only model with a worse performance for the MFPT dataset than for the CWRU dataset. The performance of RNN-WDCNN was higher than that of the 2D CNN models for the CWRU dataset but lower than that of the 2D CNN models for the MFPT dataset.

In our experiment, TICNN was the most robust against noise among the 1D CNN models. However, the test accuracy of TICNN on the MFPT dataset and noise-free environment was $97.13 \pm 0.84\%$, which was slightly lower than the accuracies of the other models. We speculated that this was because the kernel dropout rate used by TICNN was optimized only for the CWRU dataset; therefore, the performance of TICNN for the MFPT dataset in a noise-free environment was relatively low. WDCNN had a similar performance to TICNN for the MFPT dataset, but its performance declined considerably for the CWRU dataset with noise. DCN and SRDCNN exhibited comparable performances for both datasets. The accuracy of DCN was slightly higher than that of SRDCNN using the CWRU dataset with noise; however, using the MFPT dataset, SRDCNN was more robust against noise than DCN. Using both datasets with noise, these two models had lower performances than WDCNN and TICNN, which had more computations and parameters. For example, SRDCNN required 254.90 MFLOPs; thus, it performed at least 28–172 times more operations than the other 1D CNN models.

V. CONCLUSION

This study aimed to evaluate the impact of hyperparameter tuning on deep learning optimizers in bearing fault diagnosis. To this end, we selected two open-access datasets, four optimizers, and seven candidate models and performed hyperparameter tuning in two stages with varying batch sizes. Our experimental results showed the impact of hyperparameters and batch sizes on model performance.

In our benchmarking study, the learning rate and momentum factor were the key hyperparameters significantly affecting the model performance. In addition, we observed that large batch sizes led to high model performances at high learning rates or momentum factors. By contrast, the model performances were high for small batch sizes at low learning rates or momentum factors. We also evaluated the performance and computational efficiency of each model. The evaluation results showed that TICNN was the best on-device AI solution candidate in terms of computational efficiency and noise-robustness.

As a future work, we will consider the impact of optimizer weight decay or learning rate schedule. We also plan to develop a lightweight CNN model with a high noise robustness for on-device fault diagnosis.

REFERENCES

[1] S. Nandi, H. A. Toliyat, and X. Li, "Condition monitoring and fault diagnosis of electrical motors—A review," *IEEE Trans. Energy Convers.*, vol. 20, no. 4, pp. 719–729, Dec. 2005.
[2] D.-T. Hoang and H.-J. Kang, "A survey on deep learning based bearing fault diagnosis," *Neurocomputing*, vol. 335, pp. 327–335, Mar. 2019.
[3] F. Wang, J. Sun, D. Yan, S. Zhang, L. Cui, and Y. Xu, "A feature extraction method for fault classification of rolling bearing based on PCA," *J. Phys., Conf. Ser.*, vol. 628, Jul. 2015, Art. no. 012079.
[4] L. Shuang and L. Meng, "Bearing fault diagnosis based on PCA and SVM," in *Proc. Int. Conf. Mechatronics Autom.*, Aug. 2007, pp. 3503–3507.
[5] J. Zhu, T. Hu, B. Jiang, and X. Yang, "Intelligent bearing fault diagnosis using PCA–DBN framework," *Neural Comput. Appl.*, vol. 32, no. 14, pp. 10773–10781, Jul. 2020.
[6] X. Zhang, Y. Liang, J. Zhou, and Y. Zang, "A novel bearing fault diagnosis model integrated permutation entropy, ensemble empirical mode decomposition and optimized SVM," *Measurement*, vol. 69, pp. 164–179, Jun. 2015.
[7] Z. Wang, Q. Zhang, J. Xiong, M. Xiao, G. Sun, and J. He, "Fault diagnosis of a rolling bearing using wavelet packet denoising and random forests," *IEEE Sensors J.*, vol. 17, no. 17, pp. 5581–5588, Sep. 2017.
[8] X. Qin, Q. Li, X. Dong, and S. Lv, "The fault diagnosis of rolling bearing based on ensemble empirical mode decomposition and random forest," *Shock Vib.*, vol. 2017, pp. 1–9, Aug. 2017.
[9] A. Sharma, R. Jigyasu, L. Mathew, and S. Chatterji, "Bearing fault diagnosis using weighted K-nearest neighbor," in *Proc. 2nd Int. Conf. Trends Electron. Informat. (ICOEI)*, May 2018, pp. 1132–1137.
[10] Q. Lu, X. Shen, X. Wang, M. Li, J. Li, and M. Zhang, "Fault diagnosis of rolling bearing based on improved VMD and KNN," *Math. Problems Eng.*, vol. 2021, pp. 1–11, Oct. 2021.
[11] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
[12] F. Jia, Y. Lei, J. Lin, X. Zhou, and N. Lu, "Deep neural networks: A promising tool for fault characteristic mining and intelligent diagnosis of rotating machinery with massive data," *Mech. Syst. Signal Process.*, vols. 72–73, pp. 303–315, May 2016.
[13] W. Zhang, G. Peng, C. Li, Y. Chen, and Z. Zhang, "A new deep learning model for fault diagnosis with good anti-noise and domain adaptation ability on raw vibration signals," *Sensors*, vol. 17, no. 2, p. 425, Feb. 2017.
[14] W. Zhang, C. Li, G. Peng, Y. Chen, and Z. Zhang, "A deep convolutional neural network with new training methods for bearing fault diagnosis under noisy environment and different working load," *Mech. Syst. Signal Process.*, vol. 100, pp. 439–453, Feb. 2018.
[15] H. Liang and X. Zhao, "Rolling bearing fault diagnosis based on one-dimensional dilated convolution network with residual connection," *IEEE Access*, vol. 9, pp. 31078–31091, 2021.
[16] Z. Zhuang, H. Lv, J. Xu, Z. Huang, and W. Qin, "A deep learning method for bearing fault diagnosis through stacked residual dilated convolutions," *Appl. Sci.*, vol. 9, no. 9, p. 1823, May 2019.
[17] M. Zhao, S. Zhong, X. Fu, B. Tang, and M. Pecht, "Deep residual shrinkage networks for fault diagnosis," *IEEE Trans. Ind. Informat.*, vol. 16, no. 7, pp. 4681–4690, Jul. 2020.
[18] T. Chen, Z. Wang, X. Yang, and K. Jiang, "A deep capsule neural network with stochastic delta rule for bearing fault diagnosis on raw vibration signals," *Measurement*, vol. 148, Dec. 2019, Art. no. 106857.
[19] Y. Xiao, H. Shao, S. Han, Z. Huo, and J. Wan, "Novel joint transfer network for unsupervised bearing fault diagnosis from simulation domain to experimental domain," *IEEE/ASME Trans. Mechatronics*, vol. 27, no. 6, pp. 5254–5263, Dec. 2022.

[20] D. Peng, H. Wang, Z. Liu, W. Zhang, M. J. Zuo, and J. Chen, "Multibranch and multiscale CNN for fault diagnosis of wheelset bearings under strong noise and variable load condition," *IEEE Trans. Ind. Informat.*, vol. 16, no. 7, pp. 4949–4960, Jul. 2020.
[21] J. Zhao, S. Yang, Q. Li, Y. Liu, X. Gu, and W. Liu, "A new bearing fault diagnosis method based on signal-to-image mapping and convolutional neural network," *Measurement*, vol. 176, May 2021, Art. no. 109088.
[22] R. Magar, L. Ghule, J. Li, Y. Zhao, and A. B. Farimani, "FaultNet: A deep convolutional neural network for bearing fault classification," *IEEE Access*, vol. 9, pp. 25189–25199, 2021.
[23] D. Yao, H. Liu, J. Yang, and X. Li, "A lightweight neural network with strong robustness for bearing fault diagnosis," *Measurement*, vol. 159, Jul. 2020, Art. no. 107756.
[24] Y. Zhang, K. Xing, R. Bai, D. Sun, and Z. Meng, "An enhanced convolutional neural network for bearing fault diagnosis based on time–frequency image," *Measurement*, vol. 157, Jun. 2020, Art. no. 107667.
[25] Z. Zhu, G. Peng, Y. Chen, and H. Gao, "A convolutional neural network based on a capsule network with strong generalization for bearing fault diagnosis," *Neurocomputing*, vol. 323, pp. 62–75, Jan. 2019.
[26] Z. Chen, A. Mauricio, W. Li, and K. Gryllias, "A deep learning method for bearing fault diagnosis based on cyclic spectral coherence and convolutional neural networks," *Mech. Syst. Signal Process.*, vol. 140, Jun. 2020, Art. no. 106683.
[27] C.-Y. Lee, G.-L. Zhuo, and T.-A. Le, "A robust deep neural network for rolling element fault diagnosis under various operating and noisy conditions," *Sensors*, vol. 22, no. 13, p. 4705, Jun. 2022.
[28] A. Shenfield and M. Howarth, "A novel deep learning model for the detection and identification of rolling element-bearing faults," *Sensors*, vol. 20, no. 18, p. 5112, Sep. 2020.
[29] G. Jin, T. Zhu, M. W. Akram, Y. Jin, and C. Zhu, "An adaptive anti-noise neural network for bearing fault diagnosis under noise and varying load conditions," *IEEE Access*, vol. 8, pp. 74793–74807, 2020.
[30] L. Y. Imamura, S. L. Avila, F. S. Pacheco, M. B. C. Salles, and L. S. Jablon, "Diagnosis of unbalance in lightweight rotating machines using a recurrent neural network suitable for an edge-computing framework," *J. Control, Autom. Electr. Syst.*, vol. 33, no. 4, pp. 1–14, 2022.
[31] H. Fang, J. Deng, Y. Bai, B. Feng, S. Li, S. Shao, and D. Chen, "CLFormer: A lightweight transformer based on convolutional embedding and linear self-attention with strong robustness for bearing fault diagnosis under limited sample conditions," *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–8, 2022.
[32] Y. Ding, M. Jia, Q. Miao, and Y. Cao, "A novel time–frequency transformer based on self–attention mechanism and its application in fault diagnosis of rolling bearings," *Mech. Syst. Signal Process.*, vol. 168, Apr. 2022, Art. no. 108616.
[33] T. Arias-Vergara, P. Klumpp, J. C. Vasquez-Correa, E. Nöth, J. R. Orozco-Arroyave, and M. Schuster, "Multi-channel spectrograms for speech processing applications using deep learning methods," *Pattern Anal. Appl.*, vol. 24, no. 2, pp. 423–431, May 2021.
[34] N. Lopac, F. Hržić, I. P. Vuksanović, and J. Lerga, "Detection of non-stationary GW signals in high noise from Cohen's class of time–frequency representations using deep learning," *IEEE Access*, vol. 10, pp. 2408–2428, 2022.
[35] R.-Y. Sun, "Optimization for deep learning: An overview," *J. Oper. Res. Soc. China*, vol. 8, no. 2, pp. 249–294, Jun. 2020.
[36] D. Choi, C. J. Shallue, Z. Nado, J. Lee, C. J. Maddison, and G. E. Dahl, "On empirical comparisons of optimizers for deep learning," 2019, *arXiv:1910.05446*.
[37] B. T. Polyak, "Some methods of speeding up the convergence of iteration methods," *USSR Comput. Math. Math. Phys.*, vol. 4, no. 5, pp. 1–17, Jan. 1964.
[38] G. Hinton, N. Srivastava, and K. Swersky. (2012). *Neural Networks for Machine Learning Lecture 6A Overview of Mini-Batch Gradient Descent*. [Online]. Available: https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf
[39] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
[40] Y. Bengio, *Practical Recommendations for Gradient-Based Training of Deep Architectures*. Berlin, Germany: Springer, 2012, pp. 437–478, doi: 10.1007/978-3-642-35289-8_26.
[41] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *J. March. Learn. Res.*, vol. 13, no. 2, pp. 281–305, Feb. 2012.
[42] R. M. Schmidt, F. Schneider, and P. Hennig, "Descending through a crowded valley—Benchmarking deep learning optimizers," in *Proc. 38th Int. Conf. Mach. Learn.*, in Proceedings of Machine Learning Research, vol. 139, M. Meila and T. Zhang, Eds., Jul. 2021, pp. 9367–9376. [Online]. Available: https://proceedings.mlr.press/v139/schmidt21a.html
[43] P. Verma, V. Tripathi, and B. Pant, "Comparison of different optimizers implemented on the deep learning architectures for COVID-19 classification," *Mater. Today*, vol. 46, pp. 11098–11102, Jan. 2021.
[44] M. H. Saleem, J. Potgieter, and K. M. Arif, "Plant disease classification: A comparative evaluation of convolutional neural networks and deep learning optimizers," *Plants*, vol. 9, no. 10, p. 1319, Oct. 2020.
[45] S. Y. Sen and N. Özkurt, "Convolutional neural network hyperparameter tuning with Adam optimizer for ECG classification," in *Proc. Innov. Intell. Syst. Appl. Conf. (ASYU)*, Oct. 2020, pp. 1–6.
[46] B. Rezaeianjouybari and Y. Shang, "An empirical study of machine learning and deep learning algorithms on bearing fault diagnosis benchmarks," in *Proc. ASME Int. Mech. Eng. Congr. Expo.*, vol. 85611, 2021, Art. no. V07AT07A050.
[47] (2021). *Bearing Data Center*. [Online]. Available: https://engineering.case.edu/bearingdatacenter/
[48] E. Bechhoefer. (2022). *Condition Based Maintenance Fault Database for Testing of Diagnostic and Prognostics Algorithms*. [Online]. Available: https://www.mfpt.org/fault-data-sets/
[49] F. Yu and V. Koltun, "Multi-scale context aggregation by dilated convolutions," 2015, *arXiv:1511.07122*.
[50] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
[51] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 7132–7141.
[52] A. van den Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, and K. Kavukcuoglu, "WaveNet: A generative model for raw audio," 2016, *arXiv:1609.03499*.
[53] O. Bousquet, S. Gelly, K. Kurach, O. Teytaud, and D. Vincent, "Critical hyper-parameters: No random, no cry," 2017, *arXiv:1706.03200*.
[54] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 8026–8037.
[55] W. Falcon. (2019). *PyTorch Lightning*. [Online]. Available: https://github.com/Lightning-AI/lightning
[56] F. Kamalov, L. Smail, and I. Gurrib, "Stock price forecast with deep learning," in *Proc. Int. Conf. Decis. Aid Sci. Appl. (DASA)*, Nov. 2020, pp. 1098–1102.
[57] P. Nimmani, S. Vodithala, and V. Polepally, "Neural network based integrated model for information retrieval," in *Proc. 5th Int. Conf. Intell. Comput. Control Syst. (ICICCS)*, May 2021, pp. 1286–1289.
[58] P. V. Matrenin, V. Z. Manusov, A. I. Khalyasmaa, D. V. Antonenkov, S. A. Eroshenko, and D. N. Butusov, "Improving accuracy and generalization performance of small-size recurrent neural networks applied to short-term load forecasting," *Mathematics*, vol. 8, no. 12, p. 2169, Dec. 2020.



SEONGJAE LEE received the B.S. degree in mechanical and information engineering from the University of Seoul, South Korea, in 2019, where he is currently pursuing the integrated master's and Ph.D. degrees with the Department of Mechanical and Information Engineering and Smart Cities.

His current research interests include parallel computing, numerical optimization, and deep learning.



TAEHYOUN KIM (Member, IEEE) received the B.S., M.S., and Ph.D. degrees in computer engineering from Seoul National University, South Korea, in 1994, 1996, and 2001, respectively.

From 2001 to 2005, he was a Research Manager with the SoC Division, GCT Research Inc. Since 2005, he has been a Professor with the Department of Mechanical and Information Engineering, University of Seoul, South Korea. His current research interests include real-time embedded systems, application-specific GPU optimization, and on-device AI systems.